UML – Diagramas de Classes e Objetos

UNIFEI | Universidade Federal de Itajubá
Instituto de Ciências Tecnológicas
ECOI2213 – Análise e Desenvolvimento de Software II
eduardo.felipe@unifei.edu.br
fabiana.costaguedes@unifei.edu.br
rossana.junqueira@unifei.edu.br
wendelldiniz@unifei.edu.br

Versão: dd/mm/202a

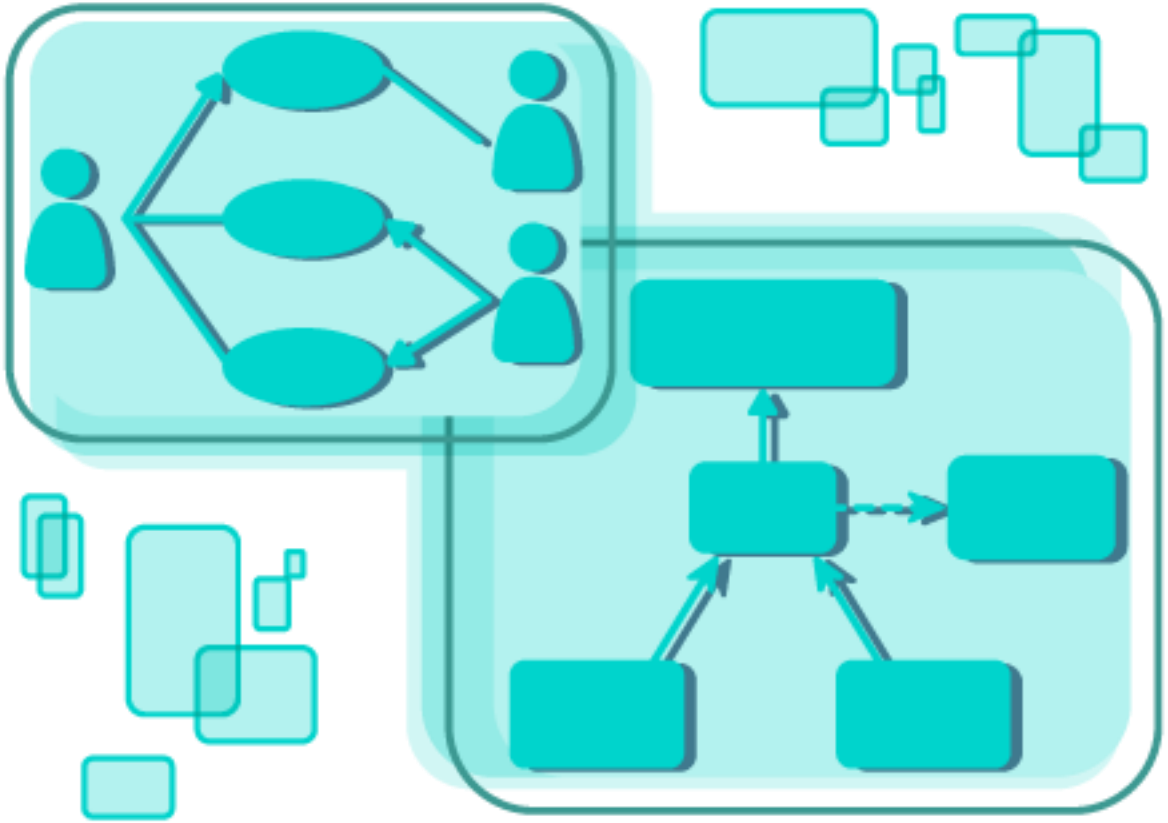


Diagrama de Classes

Unifei – Campus Itabira

Objetos

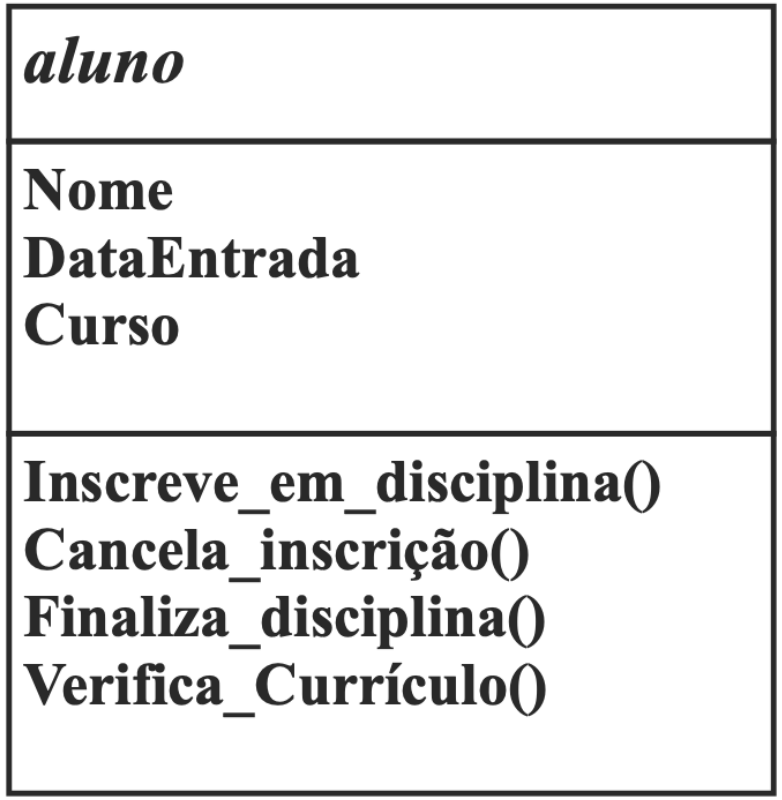
- Objetos e Classes
  - *objeto* é um *conceito*, *abstração* ou algo que tenha um significado bem definido para o problema em questão
  - uma *classe* descreve um *grupo de objetos* com o mesmo *conjunto de propriedades* (atributos), os mesmos *relacionamentos* com outros objetos e a mesma *semântica* (operações ou métodos)
- Um objeto é uma *instância* de uma classe

Diagrama de Classes

- É o diagrama central da modelagem estrutural de um sistema orientado a objetos
  - Possibilita modelar os aspectos estáticos de um sistema
- O diagrama de classes identifica
  - Classes e relacionamentos de um sistema
- O diagrama de classes não identifica
  - Sequência de interações entre objetos de um sistema

Representação de uma classe

Nome da classe



Atributos (campos)

Métodos (operações)

Representando Comportamento e Estrutura

- Uma classe engloba um conjunto de responsabilidades que definem o comportamento dos objetos desta classe.
- As responsabilidades são realizadas por meio das operações definidas para a classe.
- A estrutura de um objeto é descrita pelos atributos da classe.
- Cada atributo é uma definição de um dado que os objetos da classe devem manter.

Representações das Classes

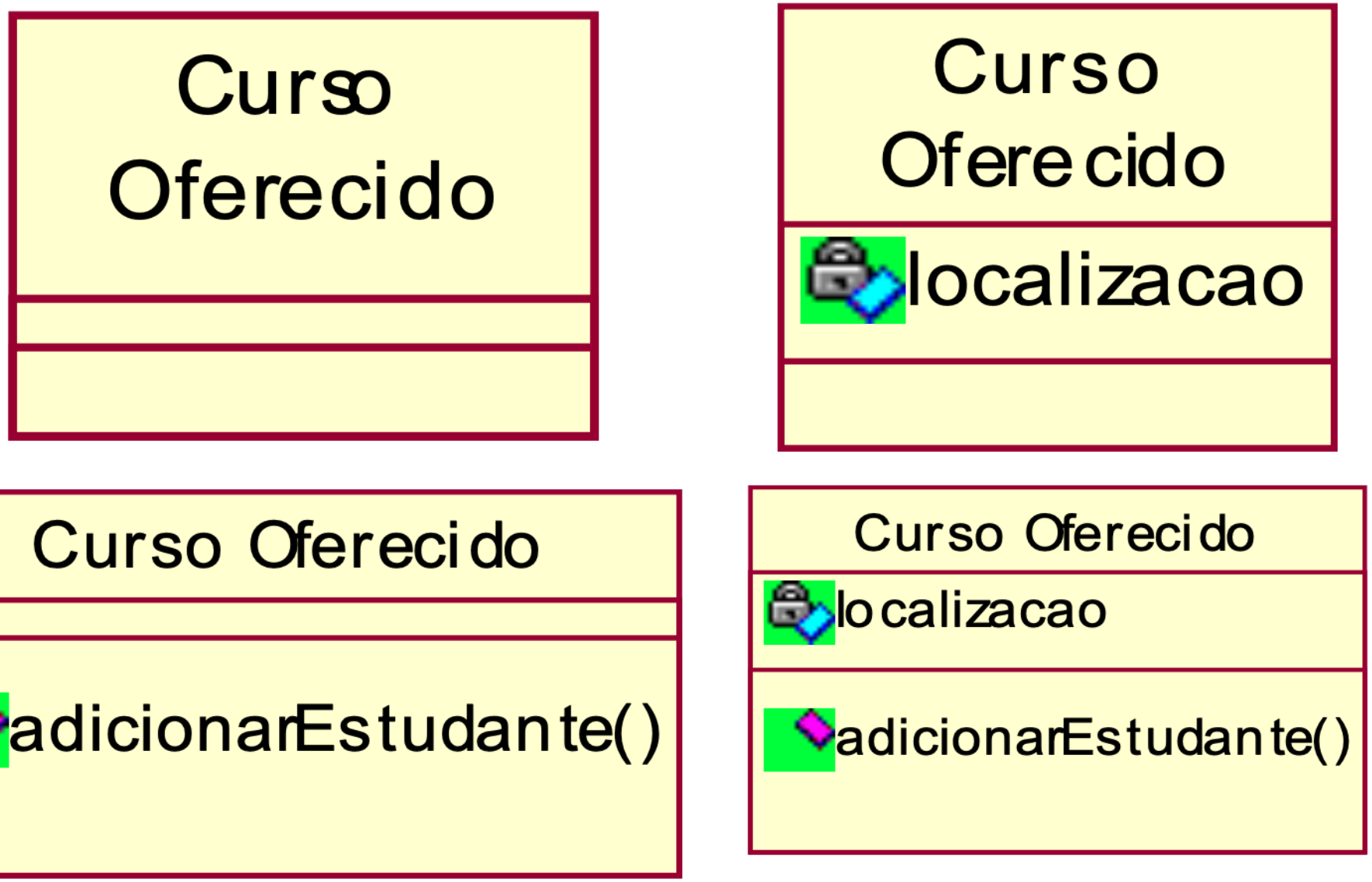
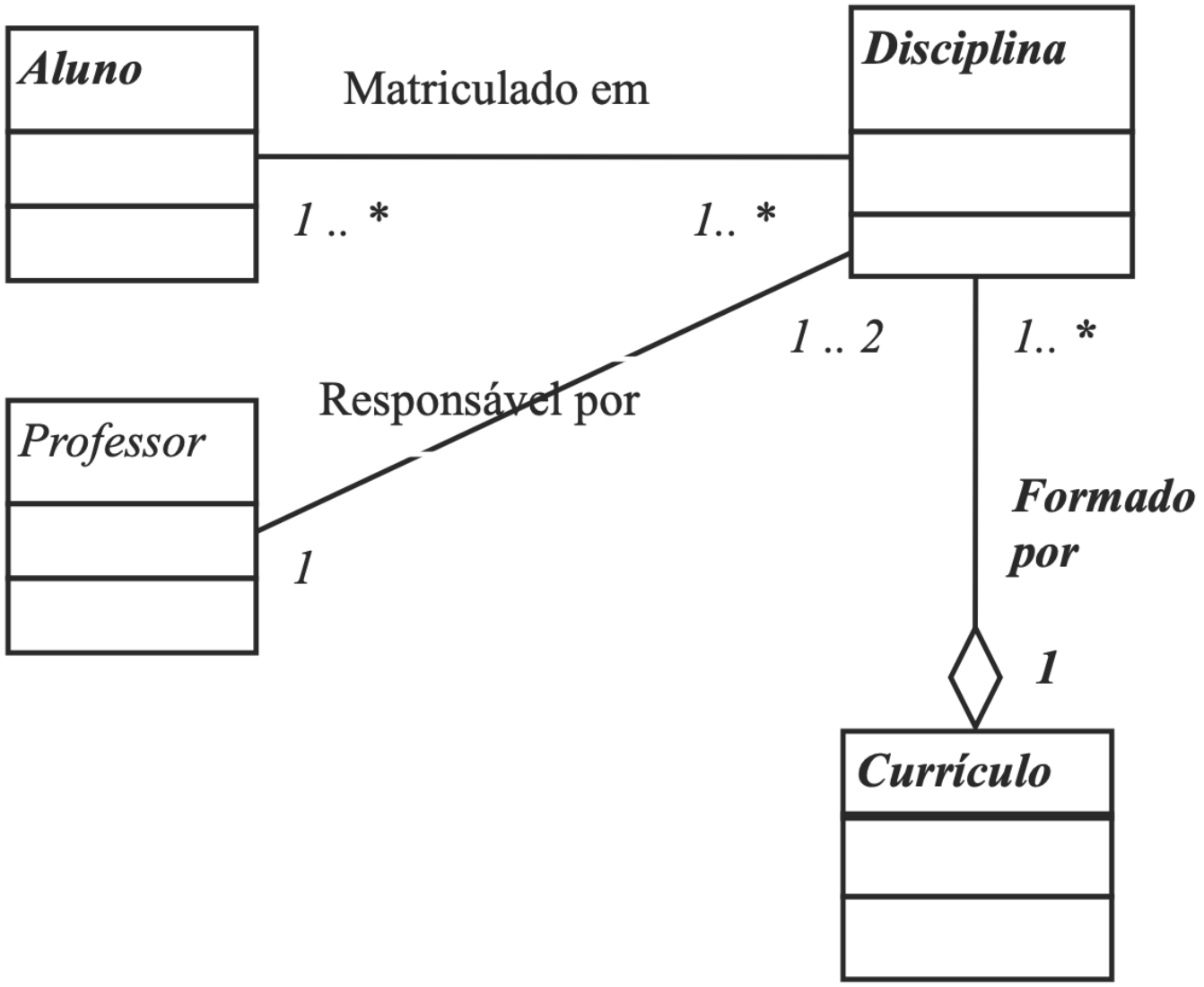


Diagrama de Classes



Exercício – Pousada – Identifique as Classes

O gerente de uma pousada deseja um sistema para gerenciar as reservas. Quando um cliente potencial deseja fazer uma reserva, o sistema verifica se existem quartos disponíveis no período, e em caso positivo, o sistema solicitará os dados do cliente (nome, endereço, telefone).

O sistema também deve armazenar sobre a reserva a data prevista para entrada, data prevista para saída, valor do desconto concedido e o número dos quartos.

Cada quarto possui um preço e uma descrição. Não há frigobar. Nem serviços de quarto.

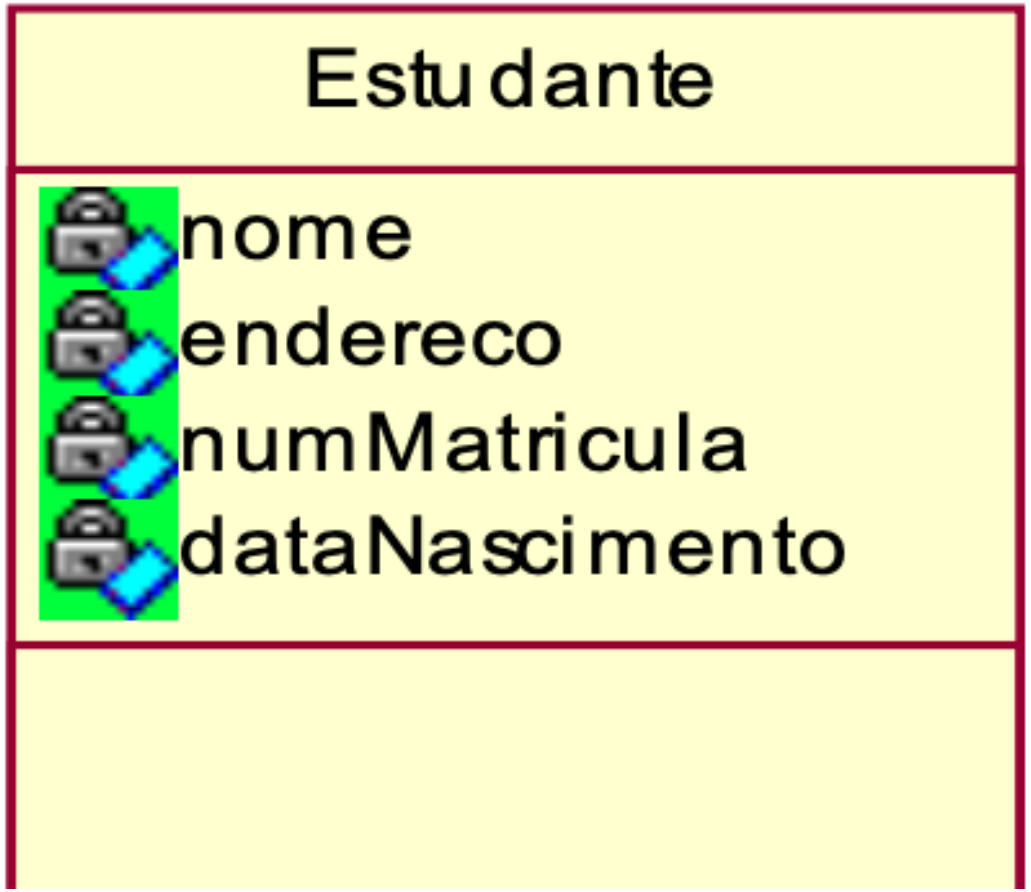
As reservas são garantidas através do pagamento de uma diária.

Caso o cliente não efetue este pagamento até três dias antes da data prevista de entrada, a reserva é cancelada pelo sistema. Um relatório de reservas canceladas é gerado pelo sistema diariamente. Outros relatórios diários são o relatório de reservas não pagas e o relatório sobre as reservas a serem efetivadas no dia.

O gerente também deseja que o sistema imprima um relatório de reservas dado um determinado período.

Atributo

- Um atributo é uma propriedade de uma classe que descreve um conjunto de valores que as instâncias da classe (objetos) podem dar para essa propriedade.
- Uma classe pode ter um número qualquer de atributos, inclusive zero.



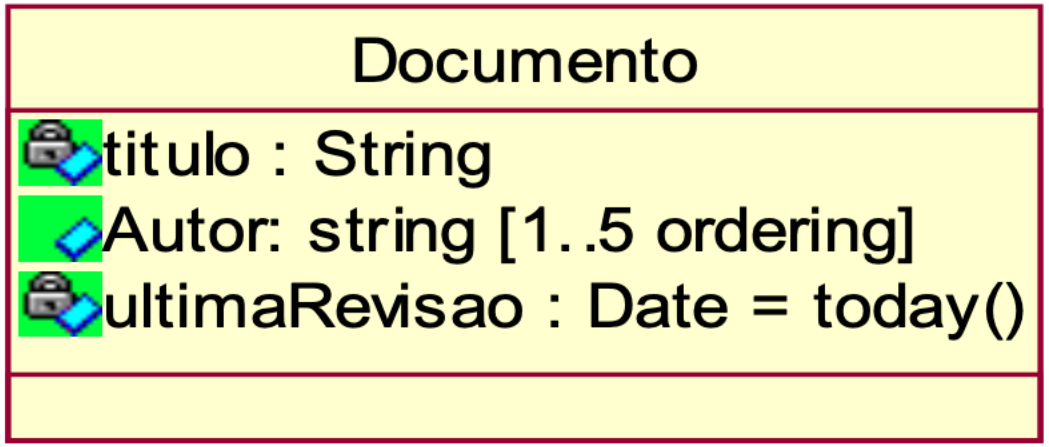
Atributo

- Sintaxe:

visibilidade nome: tipo [multiplicidade ordering] = valorInicial {propriedade}

- Exemplo

private Titulo: string
+Autor: string[1...5 ordering]
ultimaRevisão: date = hoje



Visibilidade

- Determina o nível de restrição de acesso (referência) a alguns elementos de modelagem da UML
- Aplicável às classes, atributos e métodos
- Tipos de visibilidade
  - Pública (+): pode ser referenciado por qualquer elemento de modelagem externo
  - Protegido (#): pode ser referenciado pelos elementos de modelagem internos e pelos descendentes deste (generalização)
  - Privado (-): pode ser referenciado apenas pelos elementos de modelagem internos
  - Pacote (~): pode ser referenciado por qualquer elemento de modelagem do mesmo pacote. Para elementos externos, ele é visto como privado

Propriedades

- Alguns exemplos...

{readonly}

Somente oferece operações de leitura

{ordered}, {unordered}

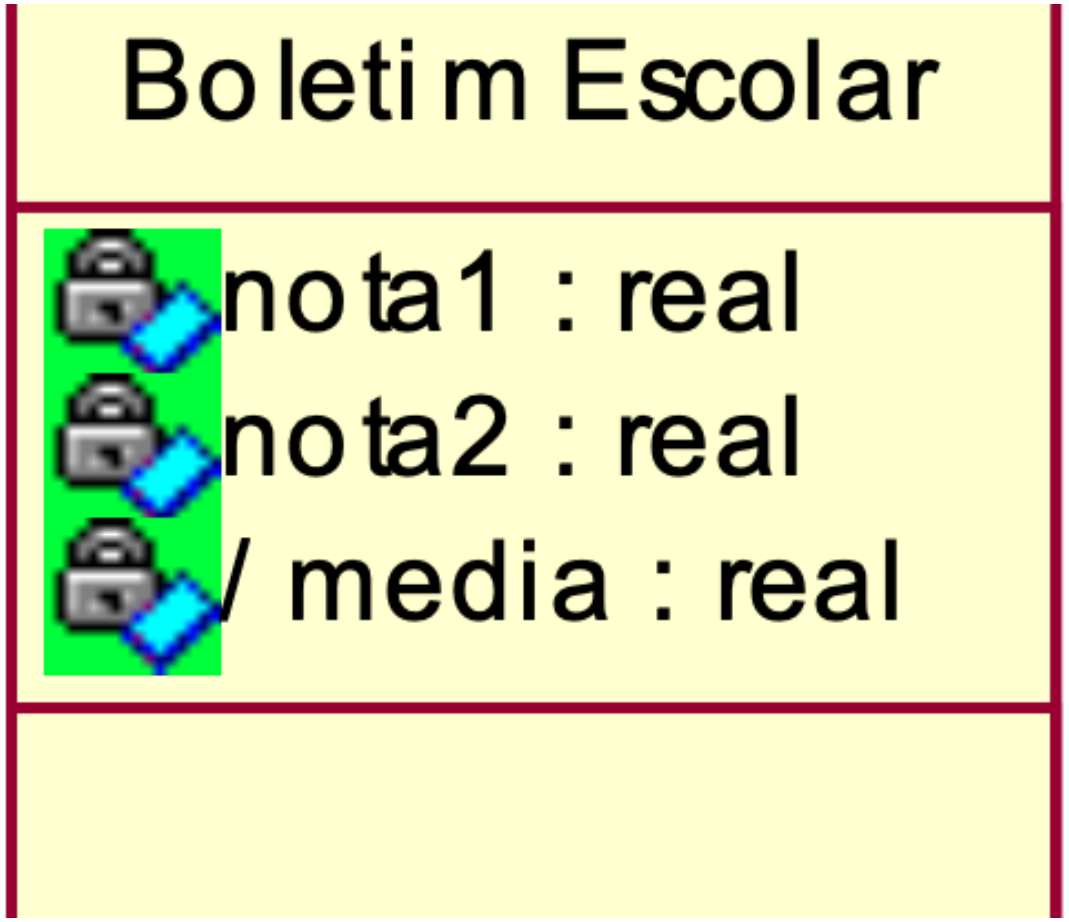
Indica se o atributo ou associação multivalorado mantém a sequência dos itens inseridos

{unique}, {nonunique}

Indica se o atributo ou associação multivalorado permite repetição

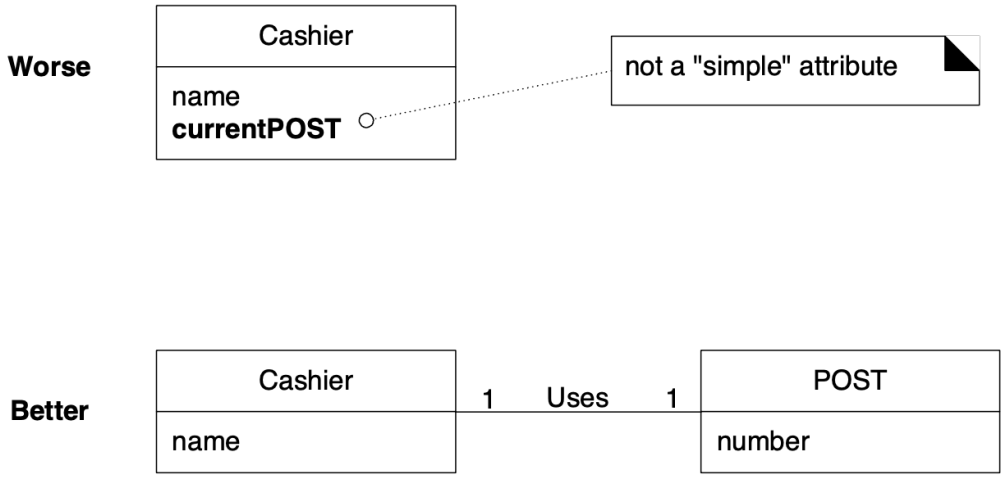
Atributo Derivado

- Um atributo *“derivado”* é um atributo cujo valor pode ser deduzido a partir de outras informações
- Usado quando não há tempo suficiente para recalcular o valor sempre que necessário.
- É uma decisão de performance X memória requerida
- Na UML, indicado com o símbolo “/”



Identificando Atributos

- Atributos devem preferencialmente representar tipos primitivos de dados ou de valores simples
  - Ex.: *Data*, *Número*, *Texto*, *Hora*, *Endereço*, etc.
- Atributos não devem ser usados para:
  - Representar uma classe complexa
  - Relacionar classes (atributo “chave-estrangeira”)



Identificando Atributos

- Podem ser representados diretamente no modelo conceitual
- Um atributo deve ser de tipo não-primitivo quando:
  - É composto de seções separadas
    - Ex.: endereço, data
  - Precisa ser analisado ou validado
    - Ex.: CPF, número de matrícula
  - Possui outros atributos
    - Ex.: Um preço promocional com prazo de validade
  - É uma quantidade com uma unidade
    - Ex.: valores monetários, medidas

Exercício – Pousada – Identifique os Atributos das Classes

O gerente de uma pousada deseja um sistema para gerenciar as reservas. Quando um cliente potencial deseja fazer uma reserva, o sistema verifica se existem quartos disponíveis no período, e em caso positivo, o sistema solicitará os dados do cliente (nome, endereço, telefone).

O sistema também deve armazenar sobre a reserva a data prevista para entrada, data prevista para saída, valor do desconto concedido e o número dos quartos.

Cada quarto possui um preço e uma descrição. Não há frigobar. Nem serviços de quarto.

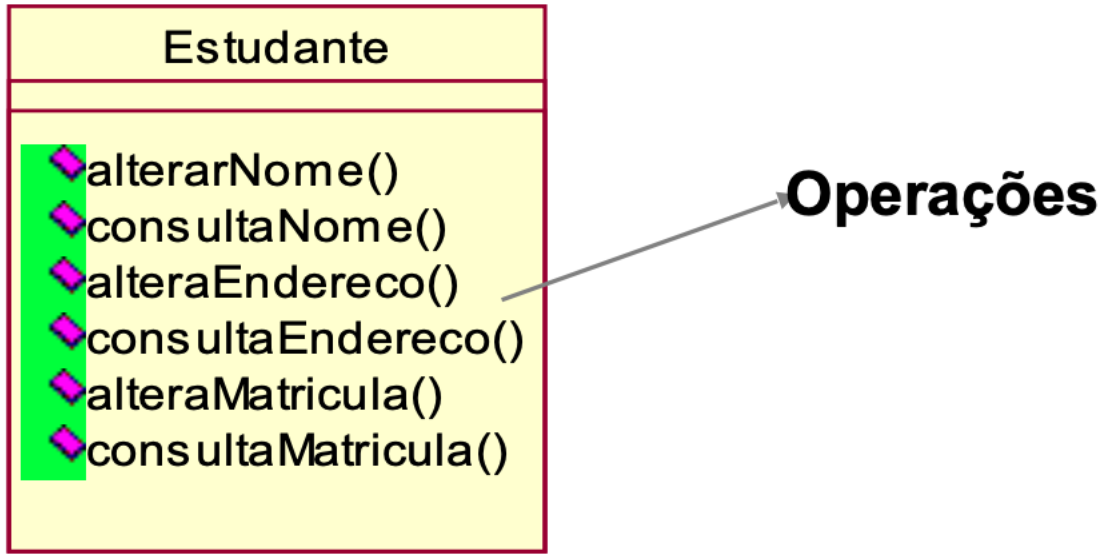
As reservas são garantidas através do pagamento de uma diária.

Caso o cliente não efetue este pagamento até três dias antes da data prevista de entrada, a reserva é cancelada pelo sistema. Um relatório de reservas canceladas é gerado pelo sistema diariamente. Outros relatórios diários são o relatório de reservas não pagas e o relatório sobre as reservas a serem efetivadas no dia.

O gerente também deseja que o sistema imprima um relatório de reservas dado um determinado período.

Operação

- Uma operação é um serviço que pode ser requisitado por qualquer objeto da classe para obter um comportamento.
- Uma classe pode ter um número qualquer de operações, inclusive zero.



Operação

- Sintaxe:

visibilidade nome (listaDeParametros): tipoDeRetorno {propriedade}

- Exemplo:

private obterSenha(): string

  - fornecerPreRequisito(codCurso: integer): ListaDeCursos

- Lista de Parâmetros (escopo nome:tipo = valorDefault)
  - in: apenas de entrada
  - out: apenas de saída
  - inout: entrada e saída (tipo default)

Propriedades de Operações

- Além das de atributos:

{query}

- Não modifica o estado do sistema após a execução

{sequential}

- A instância foi projetada para tratar uma thread por vez, mas não é sua responsabilidade assegurar que isso ocorra

{guarded}

- A instância foi projetada para tratar uma thread por vez, e é sua responsabilidade assegurar que isso ocorra (ex.: métodos synchronized em Java)

{concurrent}

- A instância é capaz de tratar múltiplas threads concorrentemente

Identificação das Operações

- Cada operação deve realizar uma função simples.
- O nome deve refletir o resultado da operação, e não as etapas.
- Evitar excesso de argumentos, necessidade de partir os métodos.
- Evitar chaves de entrada, isto é de condição para uma seleção interna à operação.
- Listar as responsabilidades da classe.
- Definir o conjunto de operações necessário para satisfazer estas responsabilidades.
- Garantir a completeza do conjunto de operações: pares abrir- fechar, incluir-excluir etc.

Nomes de Atributos e Operações

- Devem ser únicos dentro da classe.
- Nomenclatura:
  - Utilize convenções de nomenclatura para atributos e operações;
  - Promove consistência em todo o projeto;
  - Facilita a manutenção de modelos e códigos.

Exercício – Pousada – Identifique as Operações das Classes

O gerente de uma pousada deseja um sistema para gerenciar as reservas. Quando um cliente potencial deseja fazer uma reserva, o sistema verifica se existem quartos disponíveis no período, e em caso positivo, o sistema solicitará os dados do cliente (nome, endereço, telefone).

O sistema também deve armazenar sobre a reserva a data prevista para entrada, data prevista para saída, valor do desconto concedido e o número dos quartos.

Cada quarto possui um preço e uma descrição. Não há frigobar. Nem serviços de quarto.

As reservas são garantidas através do pagamento de uma diária.

Caso o cliente não efetue este pagamento até três dias antes da data prevista de entrada, a reserva é cancelada pelo sistema. Um relatório de reservas canceladas é gerado pelo sistema diariamente. Outros relatórios diários são o relatório de reservas não pagas e o relatório sobre as reservas a serem efetivadas no dia.

O gerente também deseja que o sistema imprima um relatório de reservas dado um determinado período.

A Necessidade de Relacionamentos

- As classes, no contexto dos sistemas, não trabalham sozinhas.
- As classes colaboram umas com as outras por meio de relacionamentos.
- O comportamento do sistema é obtido por meio da interação entre objetos (instâncias das Classes).

Relacionamentos entre Classes

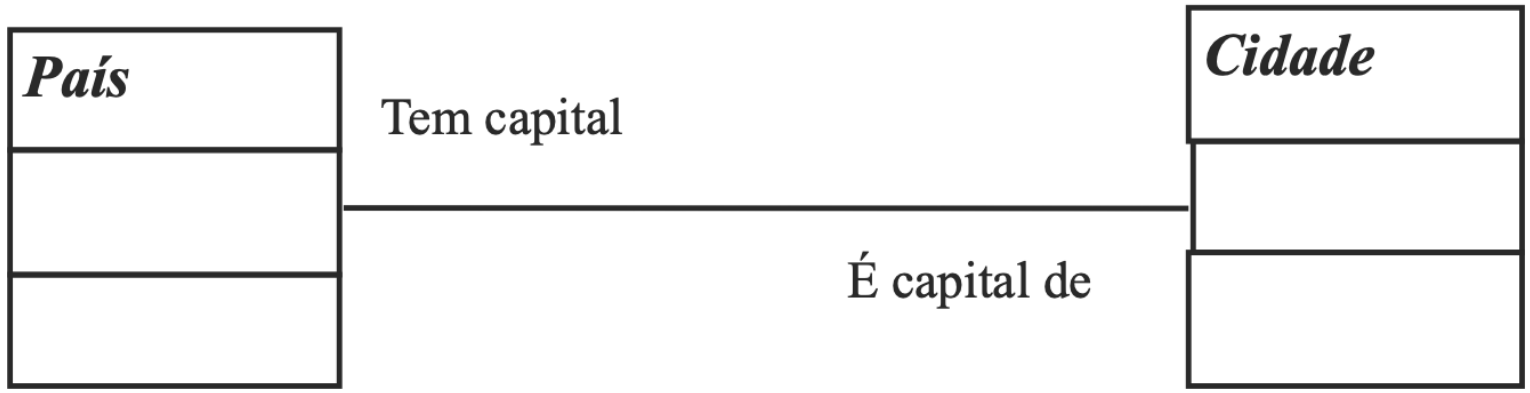
- Num sistema, as classes podem ter diversos tipos de relacionamentos entre si.
- Relacionamentos:
  - Associação;
  - Agregação e Composição;
  - Generalização;
  - Dependência;
  - Realização.

Relacionamentos: Associação

- Os objetos de uma das classes têm conhecimento dos objetos da outra:
  - Um pedido é emitido por um cliente e um cliente tem diversos pedidos pendentes.
- Conceitualmente, existe conexão semântica entre objetos das duas classes.
- Relacionamento entre classes que indica uma conexão significativa e interessante
- Descritos na UML como “relacionamentos estruturais entre objetos de tipos diferentes”
- Indicam conhecimento de um relacionamento que precisa ser preservado durante algum tempo
  - Duração de milisegundos ou anos, dependendo do contexto

Relacionamentos: Associação

- Um exemplo: supondo que *país* e *cidade* são classes num sistema, um país está associado a uma cidade que é a sua capital.



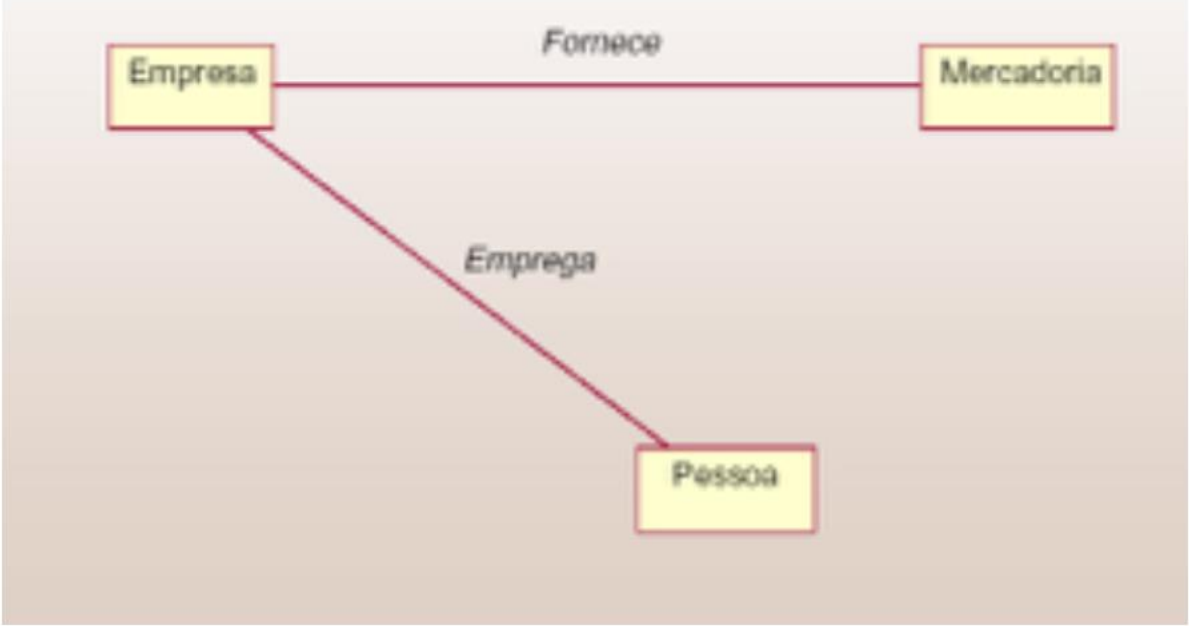
Relacionamentos e “Papéis”

- No *relacionamento* entre duas classes, cada uma desempenha um papel específico.
- Esse papel, sempre que possível, deve ser explicitado no modelo.

No exemplo anterior, *Tem capital* corresponde ao papel da classe *Pais* e *É capital de* corresponde ao papel da *Cidade* no mesmo relacionamento.

Nome da Associação

- É mostrado próximo à linha do relacionamento, de forma centralizada.
- Deve ser um nome que exprime o significado do relacionamento.
- Pode também ser um verbo, desde que esteja claro qual é o sujeito e qual é o objeto.

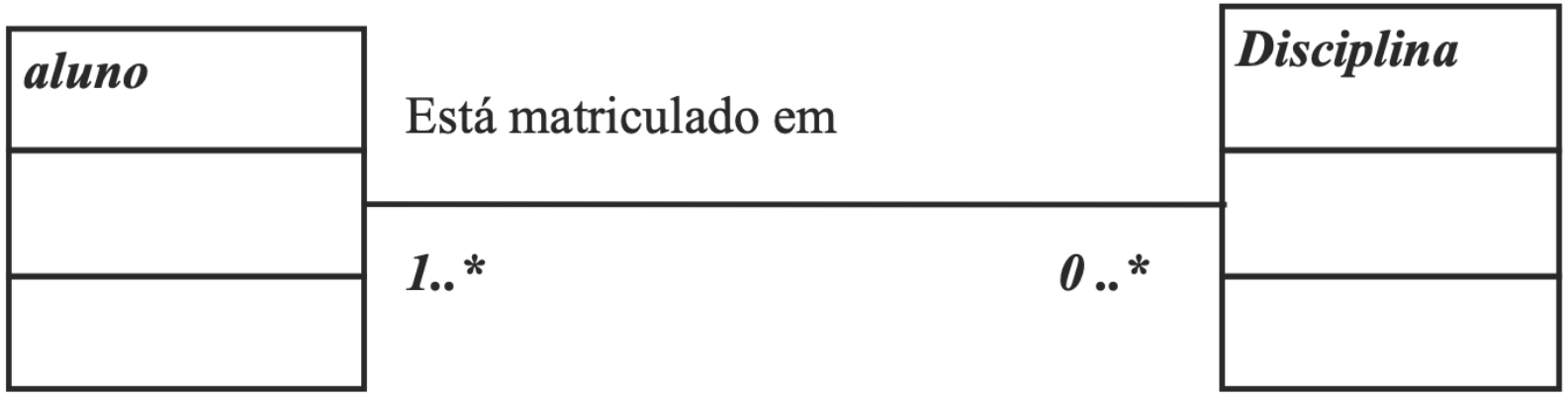


Multiplicidade

- Multiplicidade é o número de instâncias possíveis em um relacionamento entre classes.
- Consiste de um conjunto de números inteiros não-negativos.
- Se contiver um asterisco * significa uma faixa infinita de números inteiros não-negativos.

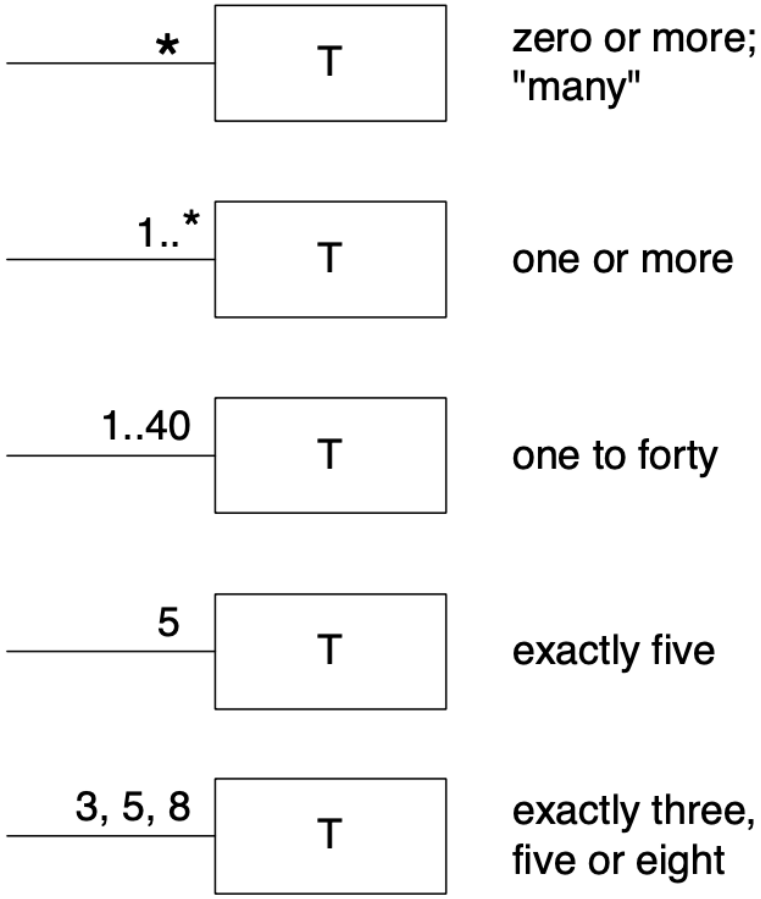
Multiplicidade

- Um exemplo: num sistema de administração escolar, *um* aluno pode se inscrever em *zero ou várias* disciplinas.



Multiplicidade

- O valor da multiplicidade depende do contexto
  - Ex.: Pessoa *Trabalha-para* Empresa



Foi eliminada na UML 2.0

Multiplicidade

- Para cada associação devem ser tomadas duas decisões de multiplicidade: uma para cada lado da associação.
- A associação entre Pessoa, atuando no papel de Professor, e Curso:
  - Para cada instância de Pessoa, um ou mais Cursos podem ser ministrados.
  - Para cada instância de Curso, exatamente uma Pessoa é o Professor

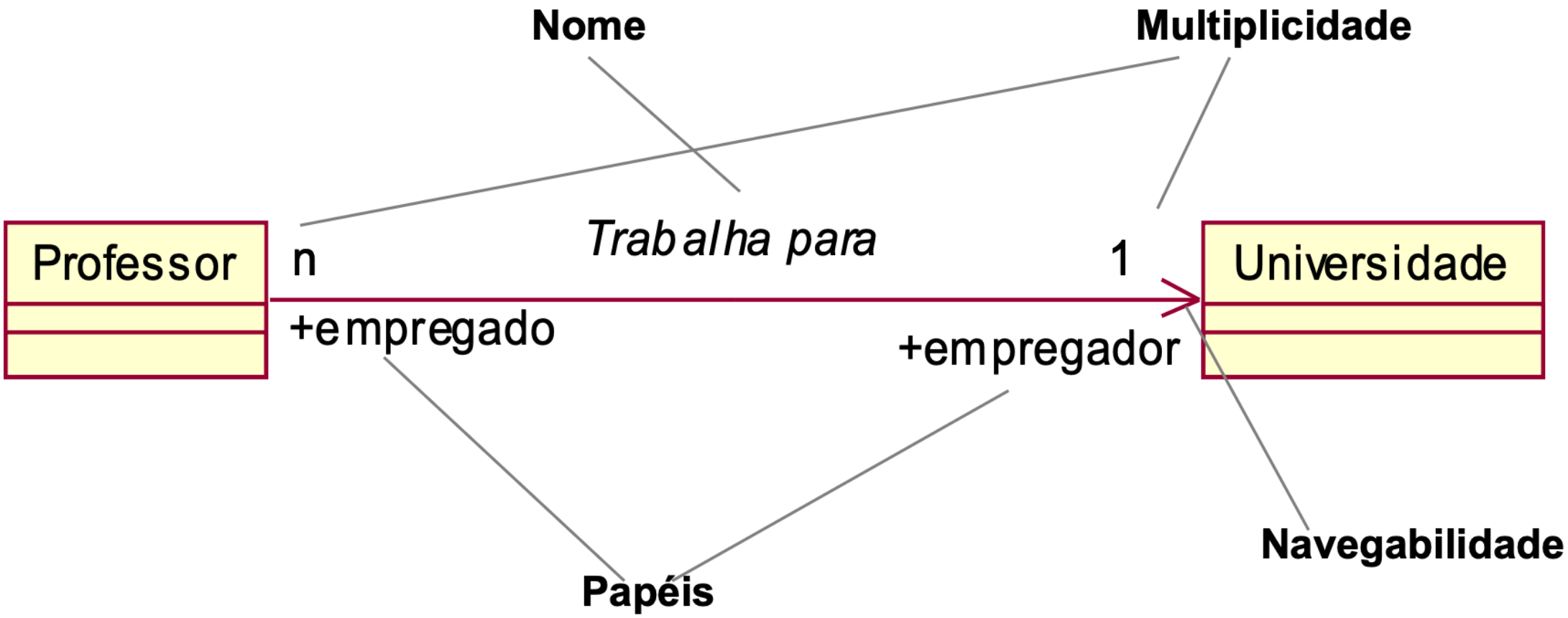
Operação

- Se uma ligação é opcional, uma operação para teste da ligação deve ser incluída.
- Exemplo:
  - Se um Professor pode estar disponível, uma operação conveniente deve ser criada na classe Professor para testar a existência da ligação.

Navegabilidade

- As associações podem ser bidirecionadas ou unidirecionais.
- Uma seta é adicionada à associação quando a navegação é em apenas uma direção.
- Quando a seta é omitida a navegabilidade é desconhecida ou é bidirecional.
- Exemplo:
  - Um eleitor deve saber em quem votou, mas o candidato não deve saber quem votou nele.
  - Deve ser implementada uma operação em eleitor para determinar seu candidato, mas não o contrário.

Nome, Papel, Multiplicidade e Navegabilidade



Exercício – Pousada – Identifique os Relacionamentos entre as Classes

O gerente de uma pousada deseja um sistema para gerenciar as reservas. Quando um cliente potencial deseja fazer uma reserva, o sistema verifica se existem quartos disponíveis no período, e em caso positivo, o sistema solicitará os dados do cliente (nome, endereço, telefone).

O sistema também deve armazenar sobre a reserva a data prevista para entrada, data prevista para saída, valor do desconto concedido e o número dos quartos.

Cada quarto possui um preço e uma descrição. Não há frigobar. Nem serviços de quarto.

As reservas são garantidas através do pagamento de uma diária.

Caso o cliente não efetue este pagamento até três dias antes da data prevista de entrada, a reserva é cancelada pelo sistema. Um relatório de reservas canceladas é gerado pelo sistema diariamente. Outros relatórios diários são o relatório de reservas não pagas e o relatório sobre as reservas a serem efetivadas no dia.

O gerente também deseja que o sistema imprima um relatório de reservas dado um determinado período.

Qualificador

- É um atributo ou lista de atributos da associação.
- Pertence à associação, não à classe.
- Seu valor serve para selecionar o conjunto de instâncias associadas com a instância do lado qualificado.
- Exemplo:
  - Para cada produto em questão pode haver a ocorrência de um item de pedido contendo o produto, ou nenhuma ocorrência.
  - Isto significa que não é possível haver dois itens de pedido com o mesmo produto para o mesmo pedido.
  - Para acessar o item é necessário identificar o produto.

Identificando Associações

- Regras úteis:
  - Focar nas associações cujo conhecimento deve ser preservado
  - Usar nomes baseados em expressões verbais que façam sentido quando lidas no contexto do modelo
  - Evitar mostrar associações deriváveis ou redundantes
  - É mais importante identificar classes do que associações
  - Associações demais tendem a confundir um modelo conceitual ao invés de iluminá-lo

Relacionamentos: Associação Reflexiva

- Indica relacionamento entre objetos de mesma classe que exercem diferentes participações.
- Não é uma associação do objeto com ele mesmo.

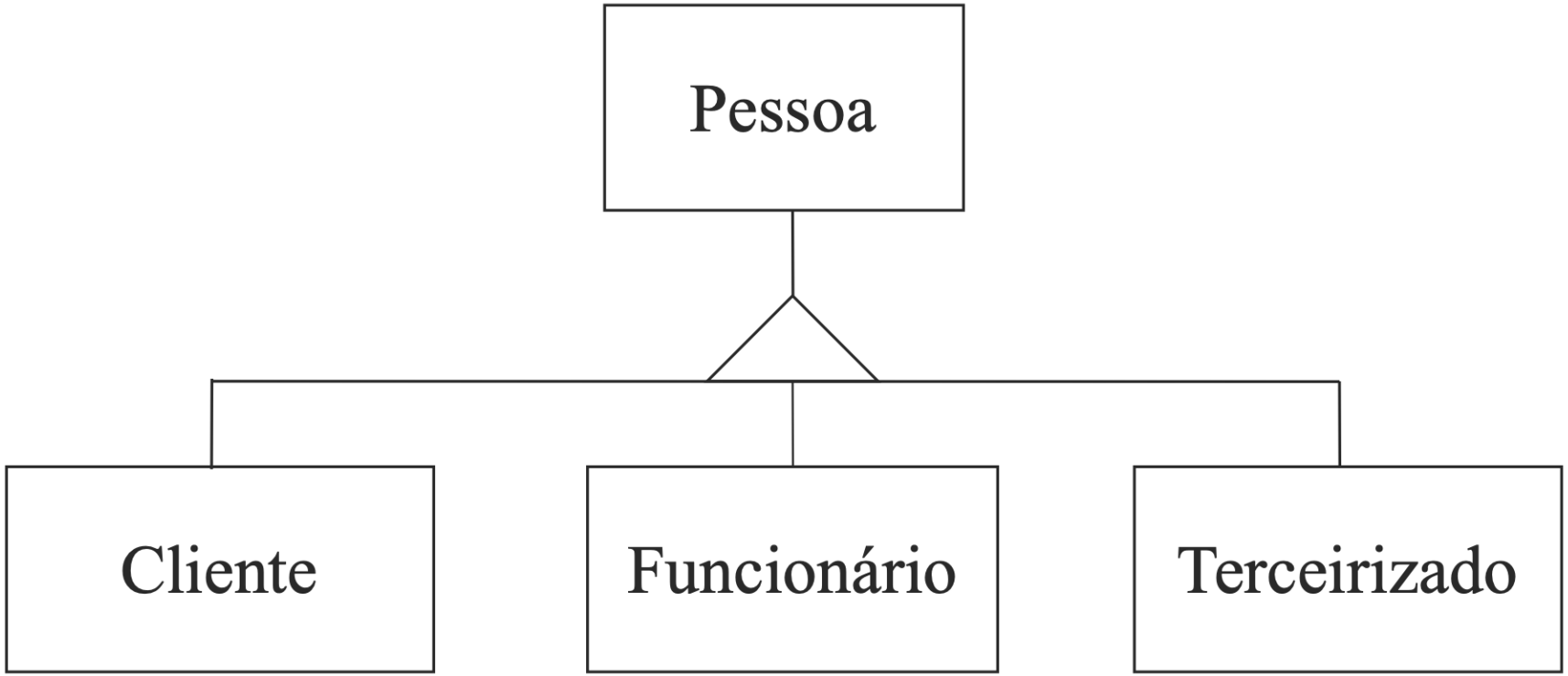
Relacionamentos: Herança / Especialização

- Uma classe pode ser derivada de outra por meio da relação de especialização.
- Essa classe herda propriedades e comportamento da segunda.
- É um relacionamento do tipo “é um tipo de”
- Generalização e herança são duas formas de abstração que permitem compartilhar semelhanças entre classes e preservar eventuais diferenças

Exemplo: A classe *Pessoa* pode ser uma generalização para as classes *Funcionário*, *Cliente* e *Terceirizado* - essas classes herdam da classe Pessoa atributos e métodos comuns

Relacionamentos: Herança / Especialização

- O relacionamento não é nomeado;
- Não se usa multiplicidade.

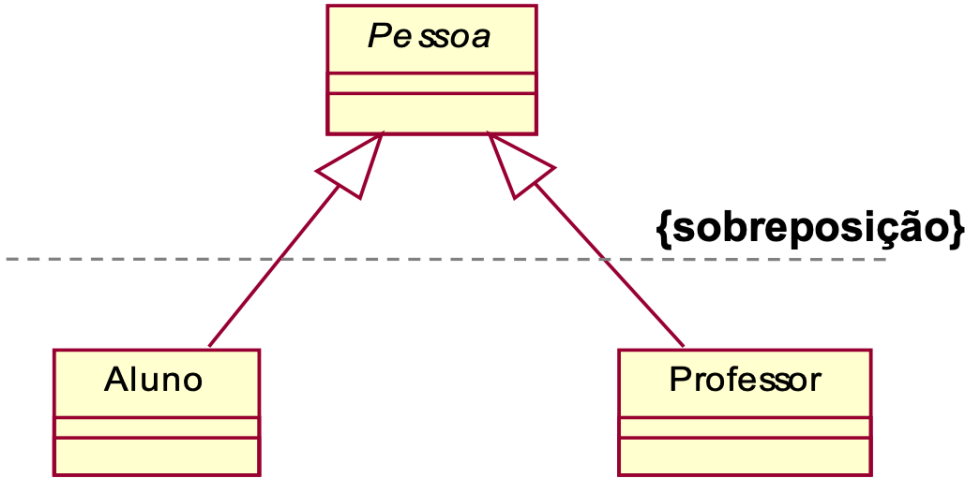


Relacionamentos: Herança / Especialização

- As subclasses herdam das superclasses:
  - atributos;
  - operações;
  - relacionamentos
- As subclasses podem:
  - acrescentar atributos e operações;
  - redefinir operações.
- Cada subclasse pode participar de relacionamentos específicos.

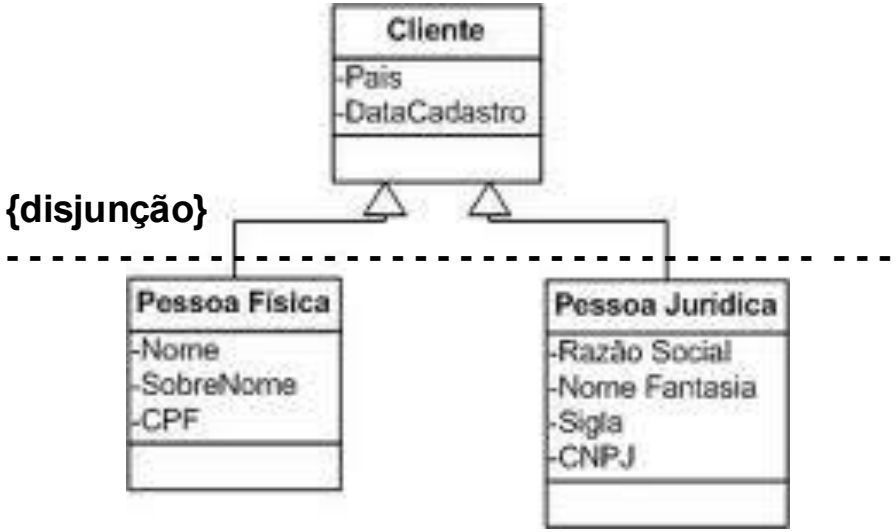
Relacionamentos: Herança / Especialização

- Sobreposição (overlapping)
  - Subclasses podem ocorrer simultaneamente em relação à mesma superclasse.
  - Exemplo: Numa universidade uma Pessoa pode ser ao mesmo tempo Aluno e Professor.



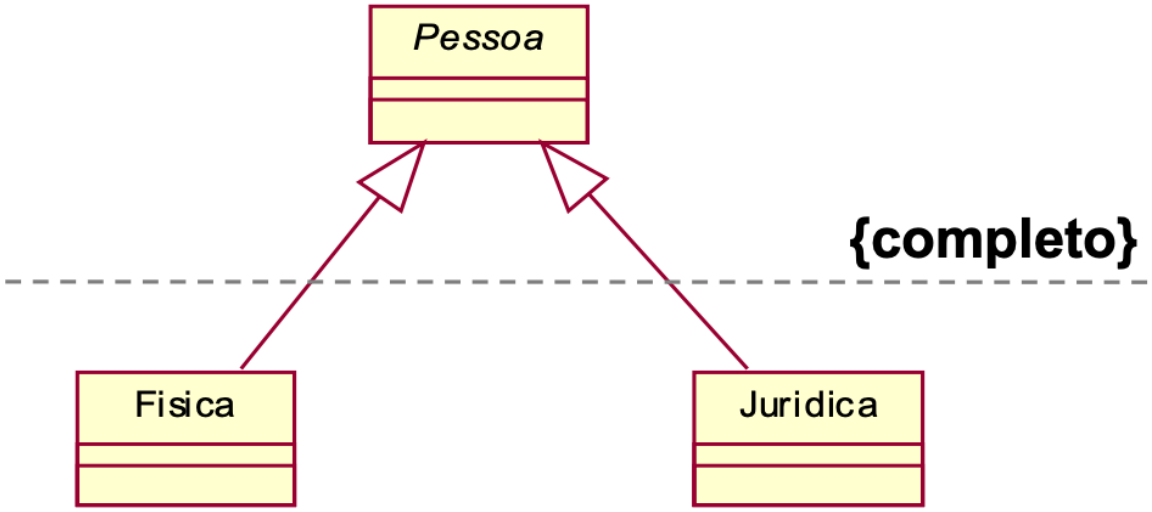
Relacionamentos: Herança / Especialização - Restrições

- Disjunção (disjoint)
  - Subclasses podem ocorrer de maneira mutuamente exclusiva em relação á mesma superclasse.
  - Exemplo: Um Cliente é pessoa física ou pessoa jurídica



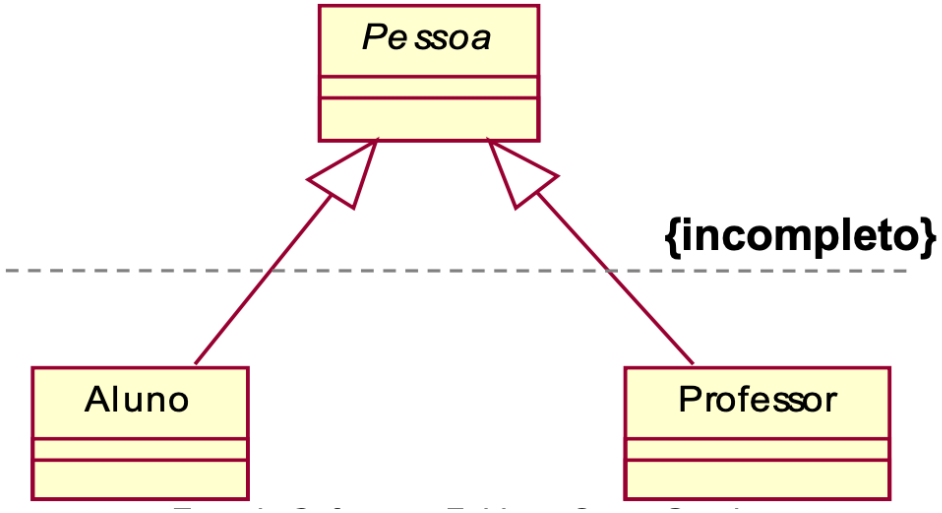
Relacionamentos: Herança / Especialização - Restrições

- Completo (complete)
  - Todas as subclasses de uma superclasse já foram especificadas e não é permitida especialização adicional.
  - Exemplo: Pessoa Física ou Pessoa Jurídica



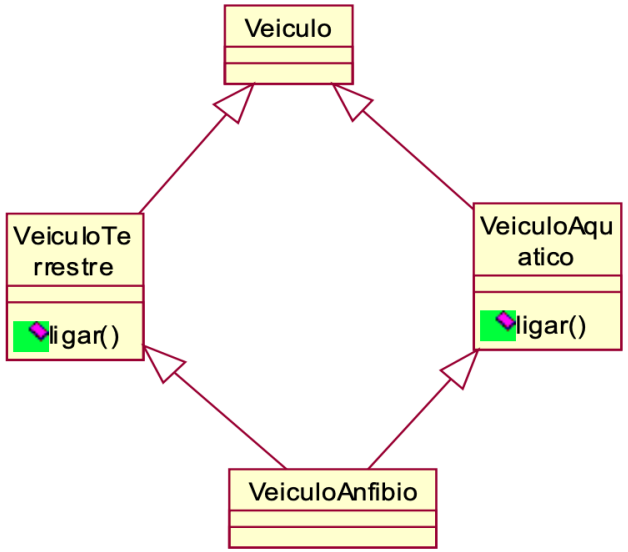
Relacionamentos: Herança / Especialização - Restrições

- Incompleto (incomplete)
  - Nem todas as subclasses possíveis foram especificadas no modelo, e inclusões ainda são permitidas.
  - Exemplo: No contexto de um sistema acadêmico outras classes tais como Funcionário Administrativo, Monitor, podem surgir.



Herança Simples e Herança Múltipla

- Herança múltipla: uma subclasse herda operações de duas ou mais superclasses.
- A herança múltipla representa múltiplos papéis de um objeto.
- Pode complicar a modelagem e deve ser usada com cuidado.
- Na UML 2 ela é representada como um conjunto de generalização. Deve-se rotular cada conjunto de generalização.
- Pode ocorrer:
  - Conflito de nome de atributo ou operação.
  - Herança repetida.

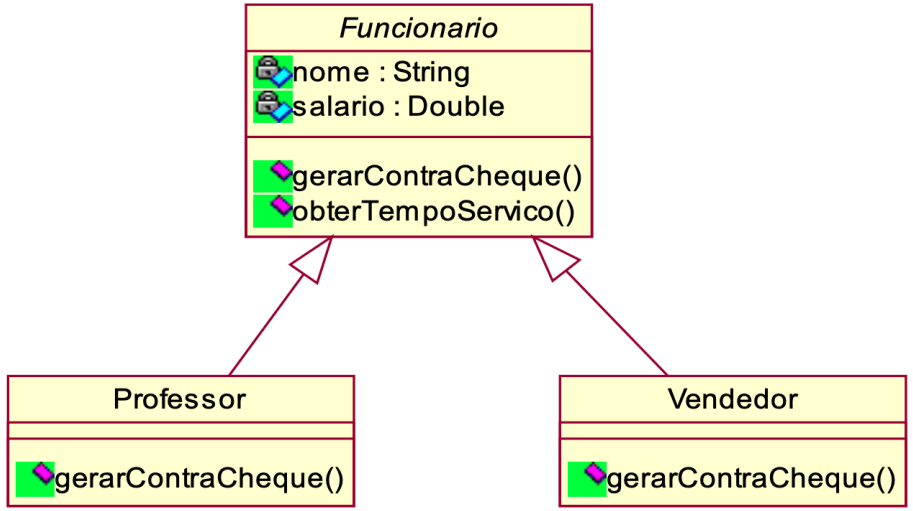


Identificação: Superclasses e Subclasses

- **Identificação das superclasses:**
  - Localização de dados ou comportamento comuns a um grupo de classes.
- **Identificação das subclasses:**
  - Localização de operações ou atributos que só se aplicam a um subconjunto das instâncias de uma classe.

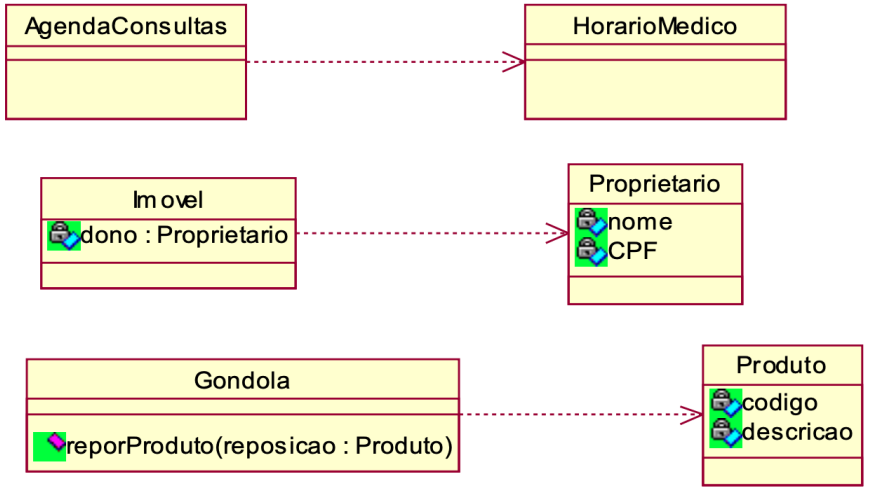
Polimorfismo

- Operações diferentes com nome comum.
  - Operações diferentes mas equivalentes.
  - Ocorre quando diferentes subclasses redefinem a mesma operação de uma superclasse.
- Pode ocorrer também entre operações da mesma classe.
- Diferenciadas pela **assinatura** (tipos dos argumentos) => Sobrecarga.



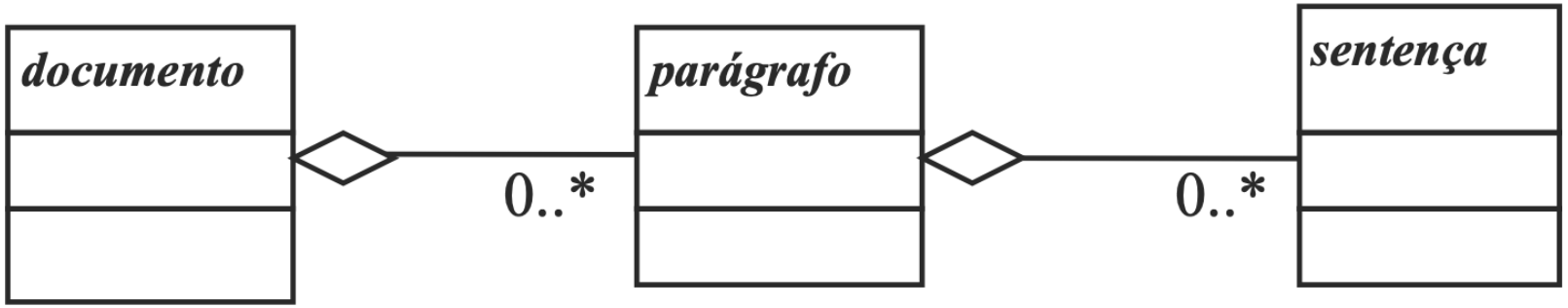
Relacionamentos: Dependência

- Indica que a mudança em uma classe pode causar mudanças na outra.
- Fatores que levam à dependência entre classes:
  - Troca de mensagem entre objetos das classes.
  - Uma classe tem como atributo outra classe.
  - Uma classe aparece como parâmetro na assinatura da operação de outra classe.



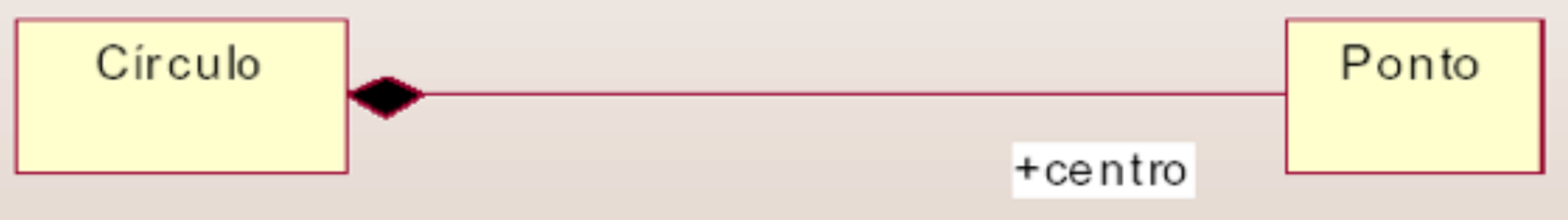
Relacionamentos: Agregação

- Descreve o relacionamento da forma “todo-parte” entre classes.
- Associações que refletem a construção física ou a posse lógica.
- Casos particulares dos relacionamentos de associação, só é necessário distinguí-los quando for conveniente enfatizar o caráter **todo-parte**.



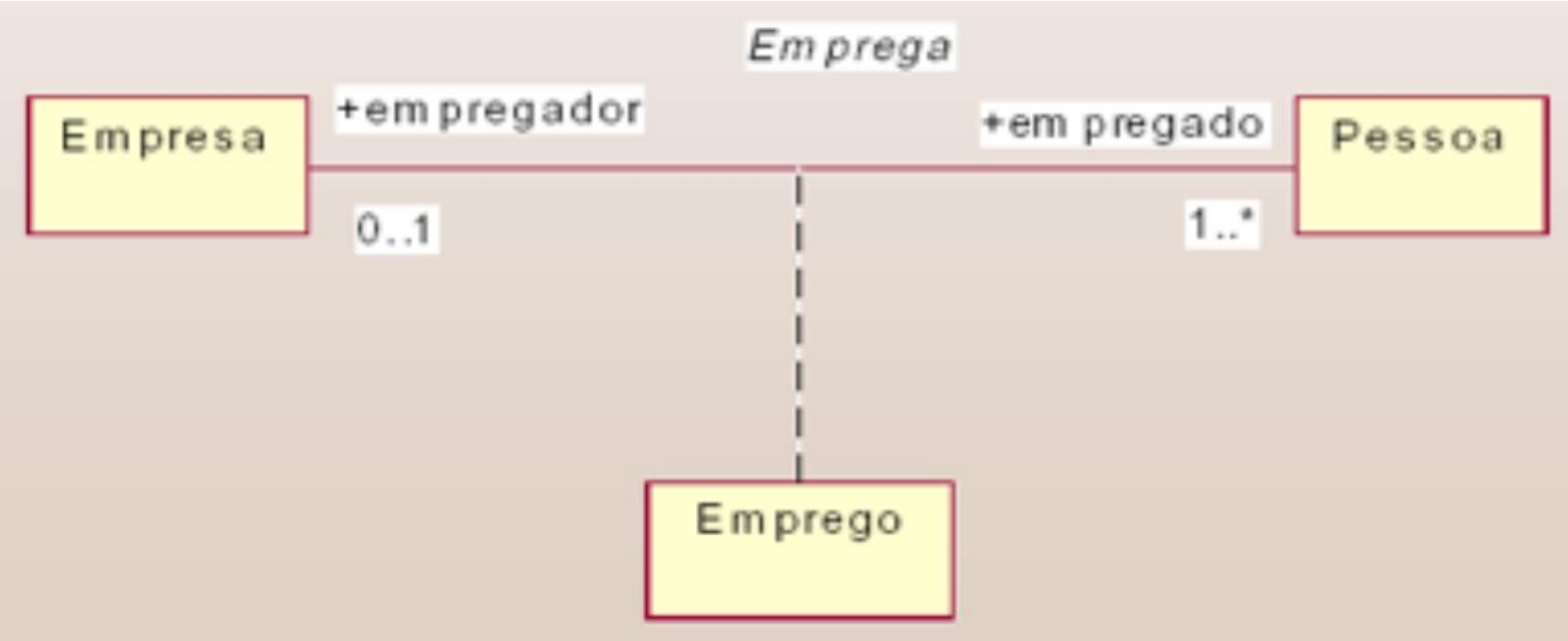
Relacionamentos: Composição

- A composição (ou agregação por composição) é uma variação mais poderosa da agregação.
- Cada parte pertence só e somente só a um todo, em um determinado momento.
- O tempo de vida das partes coincide com o tempo de vida do todo.
- As partes com multiplicidade não fixada podem ser criadas após o todo, mas uma vez criadas possuem o mesmo tempo de vida do todo.
- Estas partes também podem ser removidas antes do todo, mas não depois dele.
- Os objetos da classe parte não têm existência independente da classe todo.



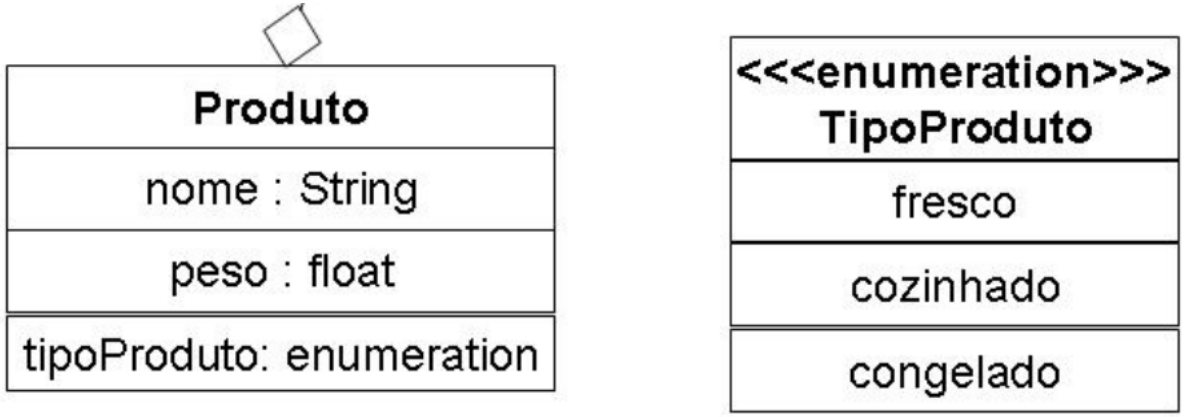
Relacionamentos: Classes de Associação

- Uma classe de associação é responsável por dados e comportamento de uma associação
- Classes que materializam relacionamentos com atributos próprios.



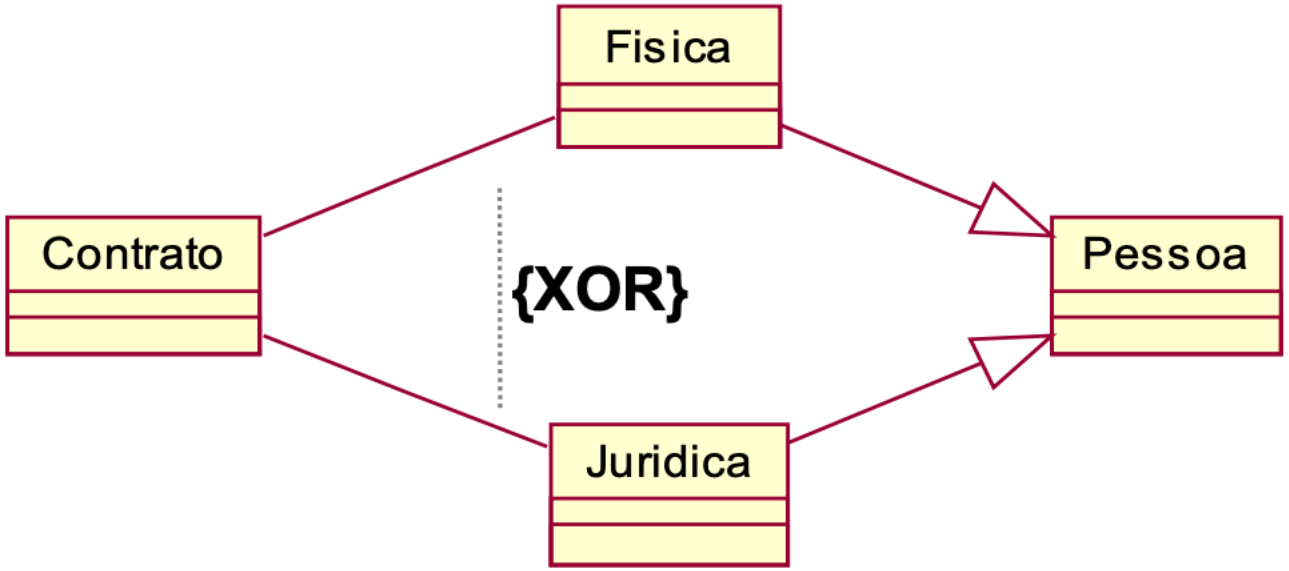
Relacionamentos: Enumeração

- Representam tipos de dados nos diagramas de classes.
- Os literais de enumeração são listados no compartimento do meio (como se fossem atributos).
- Exemplo:
  - Suponha que uma classe Produto tivesse o atributo tipoProduto.
  - Podemos usar os literais da seguinte forma:
    - Produto.tipoProduto = ‘fresco’
    - if Produto.tipoProduto = ‘congelado’ then...



Associação Exclusiva

- Uma restrição XOR indica que apenas uma dentre as várias associações representadas pode ocorrer.
- Elas não ocorrem ao mesmo tempo.



Exercício – Pousada – Identifique os Relacionamentos Especiais

O gerente de uma pousada deseja um sistema para gerenciar as reservas. Quando um cliente potencial deseja fazer uma reserva, o sistema verifica se existem quartos disponíveis no período, e em caso positivo, o sistema solicitará os dados do cliente (nome, endereço, telefone).

O sistema também deve armazenar sobre a reserva a data prevista para entrada, data prevista para saída, valor do desconto concedido e o número dos quartos.

Cada quarto possui um preço e uma descrição. Não há frigobar. Nem serviços de quarto.

As reservas são garantidas através do pagamento de uma diária.

Caso o cliente não efetue este pagamento até três dias antes da data prevista de entrada, a reserva é cancelada pelo sistema. Um relatório de reservas canceladas é gerado pelo sistema diariamente. Outros relatórios diários são o relatório de reservas não pagas e o relatório sobre as reservas a serem efetivadas no dia.

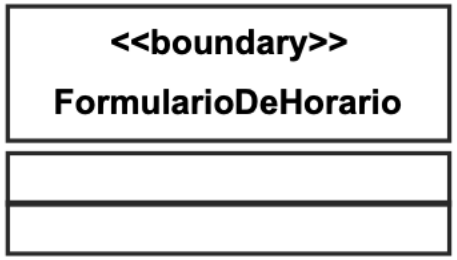
O gerente também deseja que o sistema imprima um relatório de reservas dado um determinado período.

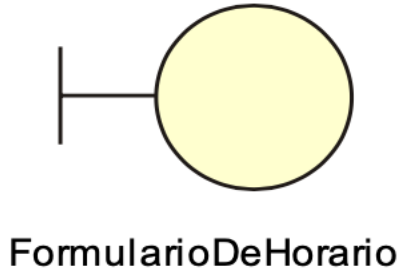
Estereótipos de Classe

- UML permite extensões pelo usuário por meio da criação de *estereótipos*.
- Na maioria dos sistemas algumas classes se encaixam em três categorias, que podem ser descritas como "*estereótipos padrão*": *Classes de Entidade*, *Classes de Controle* e *Classes de Fronteira*.

Classes de Fronteira

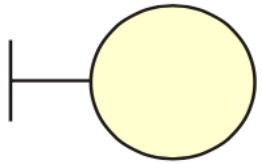
- As *Classes de Interface* descrevem as interações entre o sistema sendo descrito e o *mundo exterior*.
- Tipicamente correspondem às classes responsáveis pela *interface com o usuário* e pela *comunicação* com outros sistemas.
- Classes de fronteira típicas:
  - janelas e relatórios (interface com o usuário);
  - protocolos de comunicação (interface com o sistema);
  - interface com dispositivos de hardware;
  - sensores.



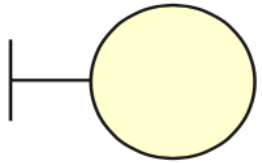


Descobrindo Classes de Fronteira

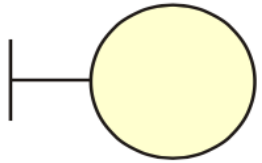
- Uma classe de fronteira representa a necessidade de comunicação entre elementos externos ao sistema com seus elementos internos.
- Toda informação trocada entre um ator e o sistema está contida na classe de fronteira.
- Analisar cada ator / caso de uso.
- No caso de interface com o usuário (GUI, relatórios), suas responsabilidades normalmente são:
  - Capturar eventos e dados do mundo externo;
  - Apresentar informações ao mundo externo.
- Quando o ator representa um outro sistema, uma classe de fronteira é utilizada para modelar a interface com o outro sistema



Tela de Mercadorias



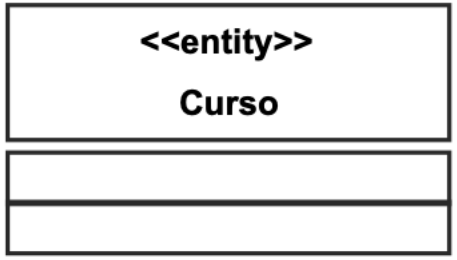
Tela de Usuários

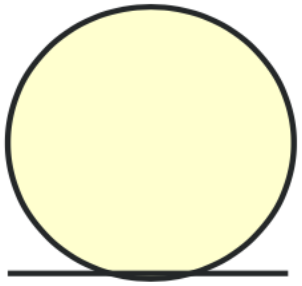


Tela de Nota Fiscal

Classes de Entidade

- Normalmente são identificadas com os principais elementos no domínio do problema.
- Necessárias para cumprir alguma responsabilidade do produto.
- *Classes de Entidade*, na maioria dos casos, se referem a informações que devem ser mantidas em bancos de dados (persistentes).





Descobrindo Classes de Entidade

- Substantivos existentes nos fluxos dos casos de uso.
- Identificar coisas tangíveis e papéis que estas desempenham.
- Identificar as **responsabilidades**: ações que possibilitam às classes cumprir seu papel nos casos de uso.
- Pessoas ou organizações que atuam como participantes no sistema e que o sistema precisa conhecer:
  - Cliente, Agência, Empresa.
- Modele uma classe para cada papel que uma pessoa possa atuar caso o sistema precise das informações:
  - Cliente, Fornecedor, Funcionário, Estudante, OperadorDaMaquina.

Descobrindo Classes de Entidade

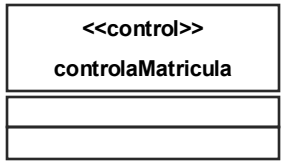
- Coisas tangíveis:
  - Produto, Fita, Filme, Ramal, Linha
- Transações:
  - Autorização, Venda, Reserva, Empréstimo, Remessa, Pedido, Pagamento
- Observe as transações e suas contrapartidas:
    - Venda X Pagamento X Cobrança.
- Outras abstrações relevantes ao domínio do problema (associados a outro objeto):
  - Endereço --> Pessoa
  - Tipo de Combustível --> Automóvel
  - Cor --> Tela

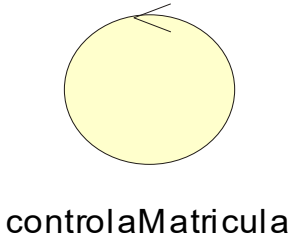
Descobrindo Classes de Entidade

- Coleções:
  - Veja cada objeto como um elemento de um conjunto. Dê um nome para o conjunto:
    - Sócio --> Lista de Sócios ou Associados.
  - Veja cada objeto como um conjunto:
    - Departamento --> Professor.
- O objeto coleção terá responsabilidade de manutenção sobre os objetos da coleção.

Classes de Controle

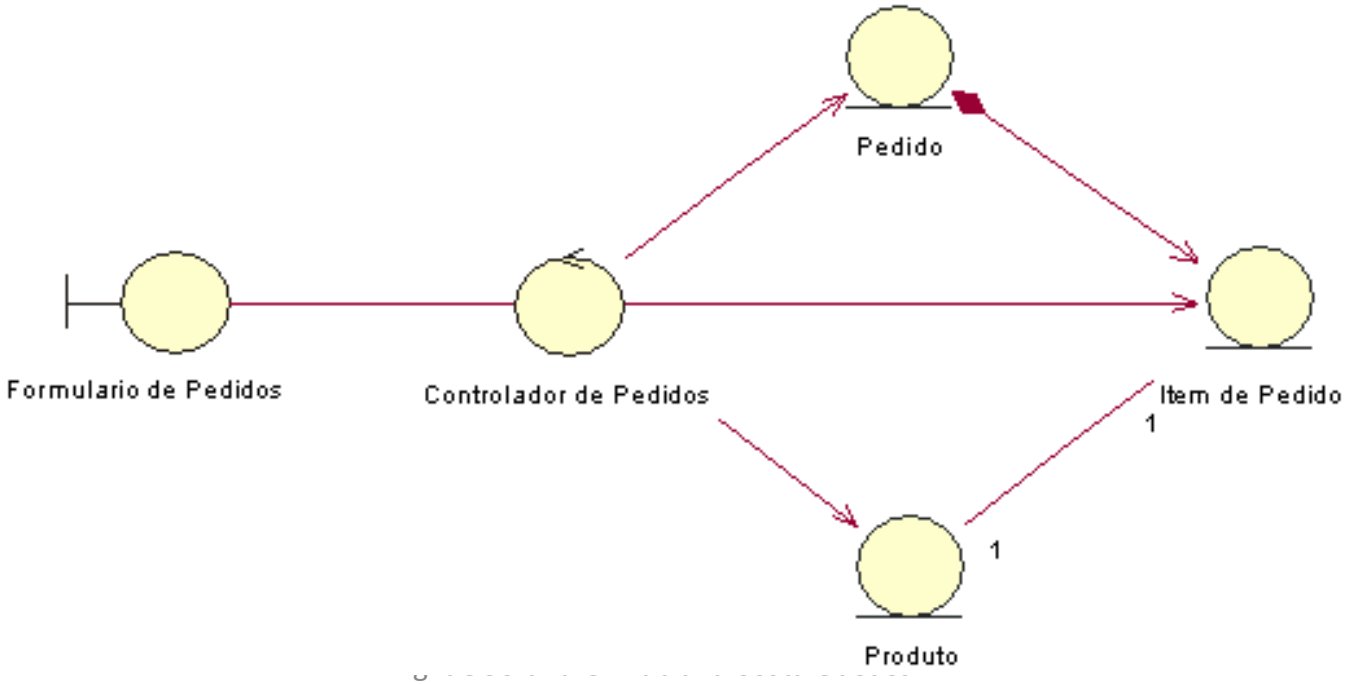
- As Classes de Controle normalmente são responsáveis pela coordenação das atividades especificadas nos casos de uso e aparecem como participantes nos Diagramas de Interação.
- São classes dependentes da aplicação, responsáveis pelas “regras de negócio” envolvidas.
- Modelam o comportamento de controle específico a um ou mais casos de uso;
- Cria, inicializa e exclui objetos controlados;
- Coordena a ação dos objetos controlados.
- Exemplos:
  - Controlador de conexão; Controlador de impressão; controlador de matrícula; Controlador de Pedido; Controlador de faturamento.





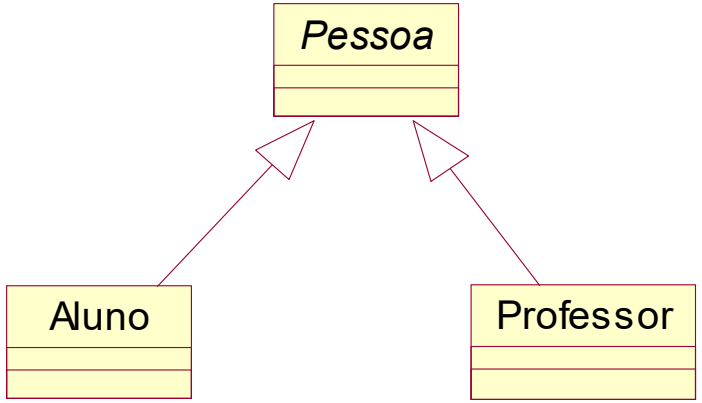
Descobrindo Classes de Controle

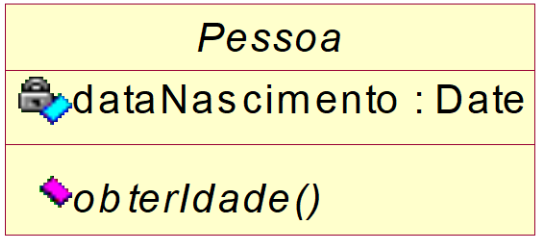
- Classes de Controle:
  - Contém informações transacionais que envolvem mais de uma classe de entidade.



Classe Abstrata

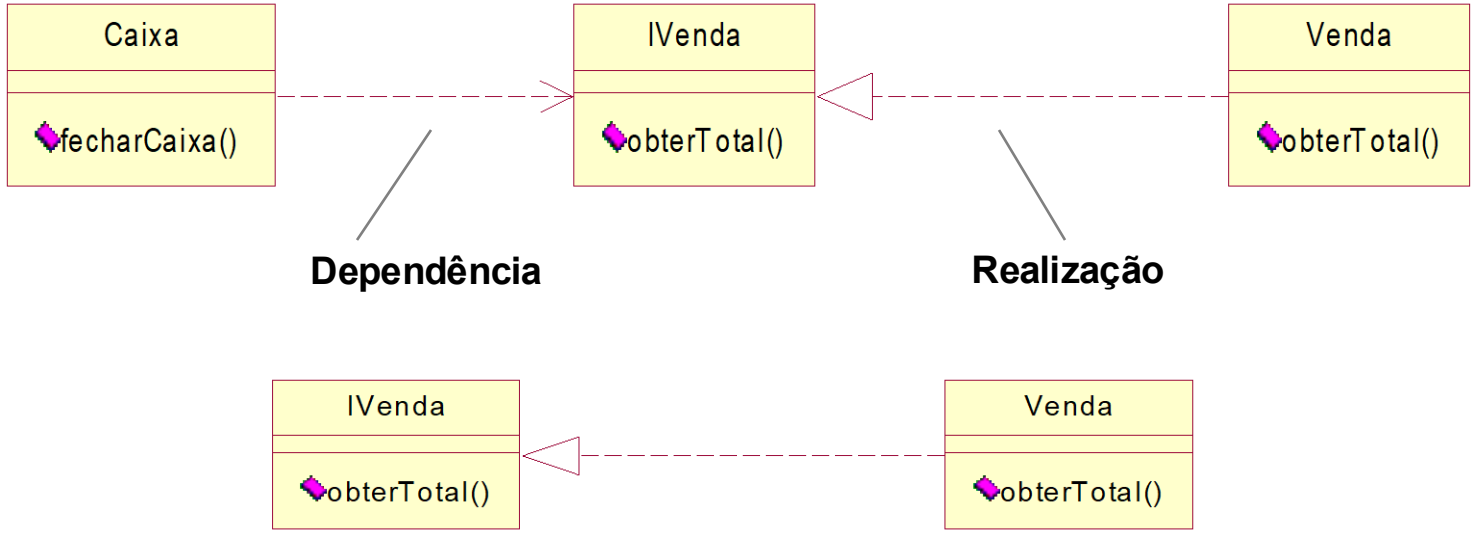
- Classes abstratas não têm instâncias.
- Têm apenas subclasses.
- Servem para capturar características comuns a grandes grupos de objetos.
- As operações apenas definem interfaces obrigatórias para as subclasses.
- Por default uma operação é considerada como não-abstrata.
- Uma operação abstrata é aquela que não possui um método que a implemente.
- Uma classe que possui uma ou mais operações abstratas é abstrata.





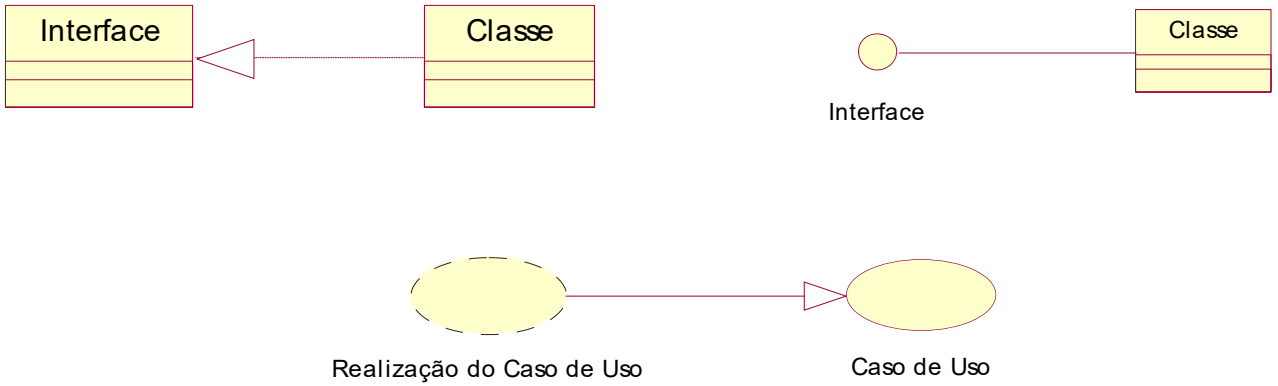
Classe de Interface

- O propósito de uma classe de interface é coletar um conjunto de operações oferecidas pela classe.
- É comum apresentarmos na interface apenas parte das operações.
- A interface especifica a assinatura das operações.
- O relacionamento utilizado é o de realização.
- Uma outra classe que use ou requeira operações suprimidas pela interface é ligada a esta por uma dependência.



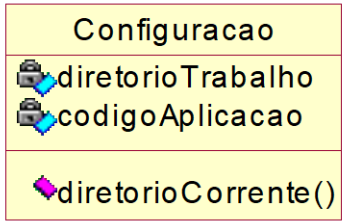
Realização

- Um elemento serve de “contrato” com o qual outros elementos concordam em realizar.
- Encontrado entre:
  - Interfaces e os elementos que as realizam (implementem).
  - Casos de uso e as colaborações que os realizam



Classe Utilitária

- Um utilitário agrupa variáveis e procedimentos globais na forma de uma classe.
- Em geral nenhuma instância é especificada, logo os atributos e operações são declarados com escopo de classe.
- Exemplo:
  - Um sistema que precise guardar informações do tipo diretório de trabalho, código da aplicação em execução, e outras configurações.

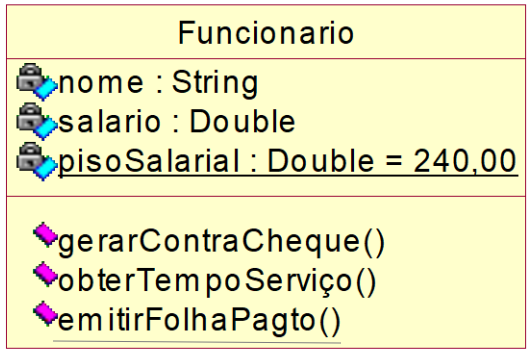


Escopo

- Escopo de instância:

objetoP.nome --> Pedro
objetoP.salario --> 500,00
objetoP.gerarContraCheque

objetoA.nome --> Augusto
objetoA.salario --> 2000,00
objetoA.gerarContraCheque
É necessário instanciar os objetos.

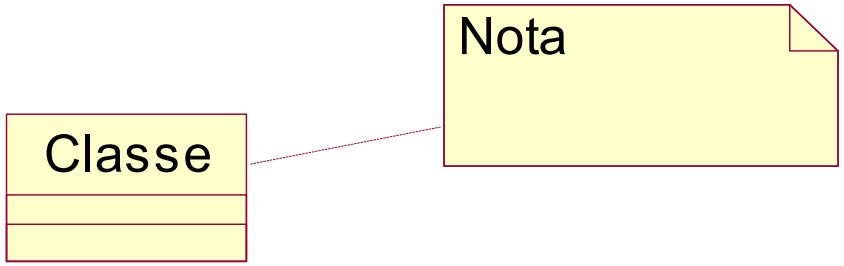


- Escopo de classe:

Funcionario.pisoSalarial
Funcionario.emitirFolhaPagto()
Não é necessário instanciar um objeto.

Nota

- Nota é um comentário que pode ser adicionado a um diagrama para incluir mais informação.
- Pode ser adicionada a qualquer elemento UML.
- Deve ser ancorado a um elemento com uma linha pontilhada.

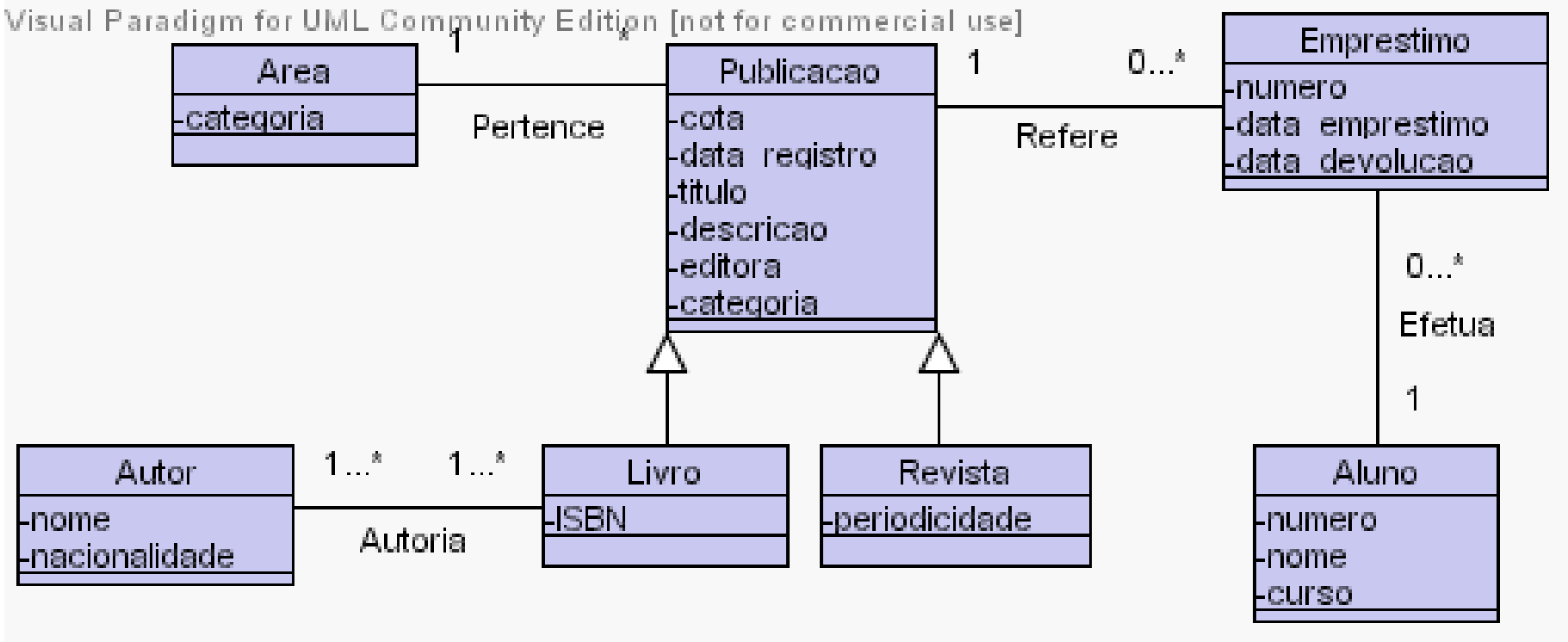


Exercício: Biblioteca

- Considere a seguinte informação adicional à descrição apresentada no exercício da Biblioteca. Esta informação consiste num pedaço da entrevista efetuada pelo consultor Paulo Bastos ao responsável da biblioteca João Almeida.
- **Paulo Bastos:** Como é que funciona o processo de empréstimo de publicações.
- **João Almeida:** Bom, neste momento as publicações disponíveis aos alunos são os livros e as revistas que assinamos. Um aluno dirige-se com as publicações ao balcão de atendimento para preencher um ficha de empréstimo. Tem que efetuar uma ficha para cada publicação, preenchendo a cota e o título. Caso seja um livro, terá que escrever o(s) respectivo(s) autor(es).
- **P.B.:** Existe alguma limitação no número de empréstimos?
- **J.A.:** Sim, no máximo um aluno pode efetuar 3 empréstimos.
- **P.B.:** Qual é o procedimento quando chega uma nova publicação?
- **J.A.:** Bem... quando chega uma nova publicação esta é encaminhada para a responsável de catalogação onde será analisada e definida a sua área de conhecimento. Existem várias áreas pré-definidas como, por exemplo, Sociologia, Psicologia, Informática, etc. Novas áreas de conhecimento podem ser definidas.
- **P.B.:** Existe alguma informação específica sobre cada uma das publicações?
- **J.A.:** Para os livros temos que registrar o seu número de identificação internacional, ISBN, e para as revistas registramos a sua periodicidade.“
- Identifique classes e desenhe o respectivo diagrama.

Exercício: Solução

- **Identificação das classes**
  - Publicação
  - Livro
  - Revista
  - Autor
  - Área
  - Empréstimo



Diagramas de Objetos

Diagrama de Objetos

- Representa a instância de um diagrama de classes num determinado ponto do tempo. Retrato do sistema em determinado instante.
- Para cada classe temos um objeto (sua instância).
- Representação dos objetos e as ligações entre eles.
- Não descreve o sistema.

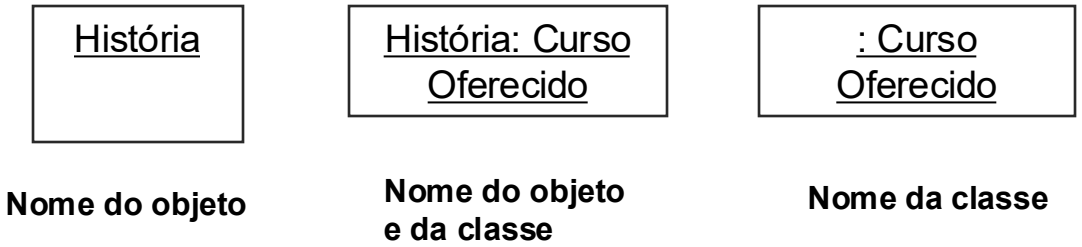
Diagrama de Objetos

- É útil para:
  - Visualizar mais claramente a relação entre os objetos das classes;
  - Visualizar dados reais;
  - Validar o modelo de classes;
  - Identificar problemas na execução de uma aplicação.

- Deve mostrar apenas os objetos relevantes em um determinado momento do tempo.
- Notação UML para objetos:
- <u>nomeDoObjeto: nomeDaClasse</u>

Diagrama de Objetos

- Representação dos objetos:



- Colocar apenas o nome da classe indica qualquer objeto da classe (objeto anônimo).

Diagrama de Objetos

- Notação UML para atributos dos objetos:
- nomeDoAtributo: tipo = valor
  - Se o tipo for citado, deve ser o mesmo da classe, mas pode ser omitido.
  - Atributos cujos valores podem mudar durante o processamento apresentam uma lista de valores.

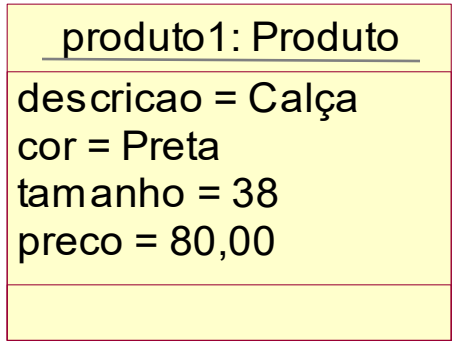


Diagrama de Objetos

- Relacionamentos entre os objetos são feitos através de links.
- Um link é uma instância de uma associação.
- O nome do papel pode ser mostrado ao final do link.
- O nome da associação pode ser mostrado próximo ao link.
- Multiplicidades não são mostradas pois os links já são instâncias.
- Agregação, composição e navegação podem ser mostradas.

Exemplo Diagrama de Objetos

- O exemplo mostra um diagrama de objetos para o cliente Michael Richardson e seus dois pedidos na Virtual LTDA. O diagrama pode ser lido da seguinte maneira: O objeto R. Michael Richardson da classe Cliente está associado a ambos os objetos 123456 e 123700 da classe Pedido.
- Usa-se o diagrama de objetos para modelar a visão estática de um sistema. Ele mostra o retrato do sistema em determinado momento.
- Retirado de: http://www.fag.edu.br/professores/elielder/materias/apoo/09.pdf

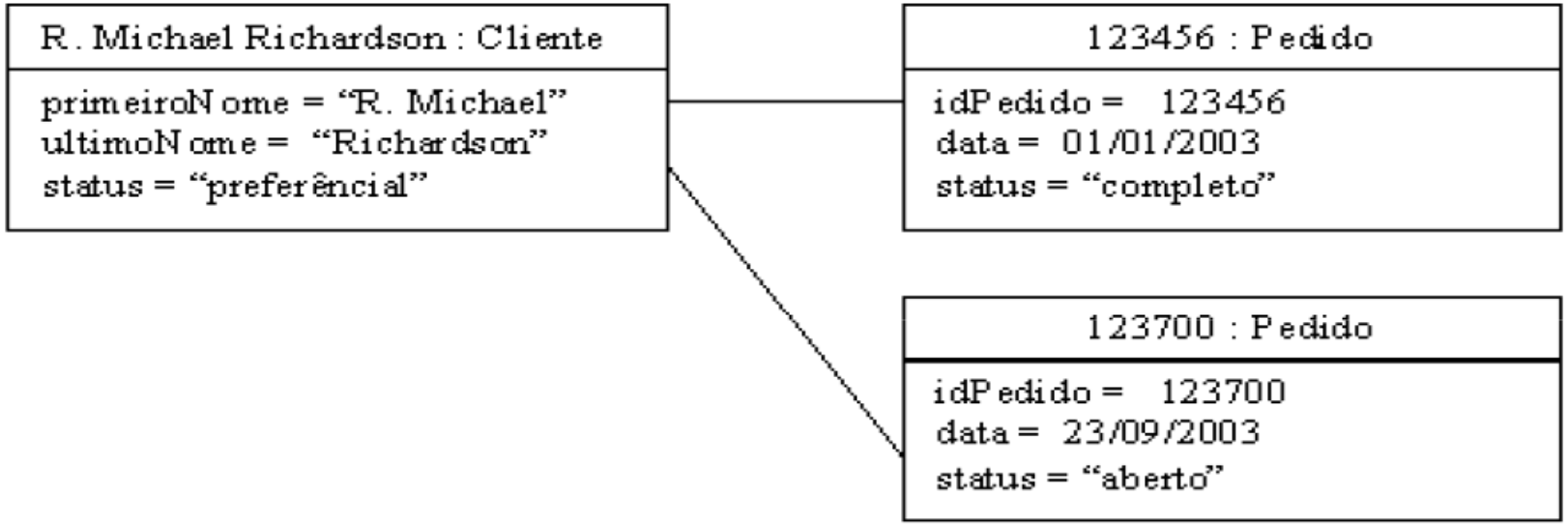
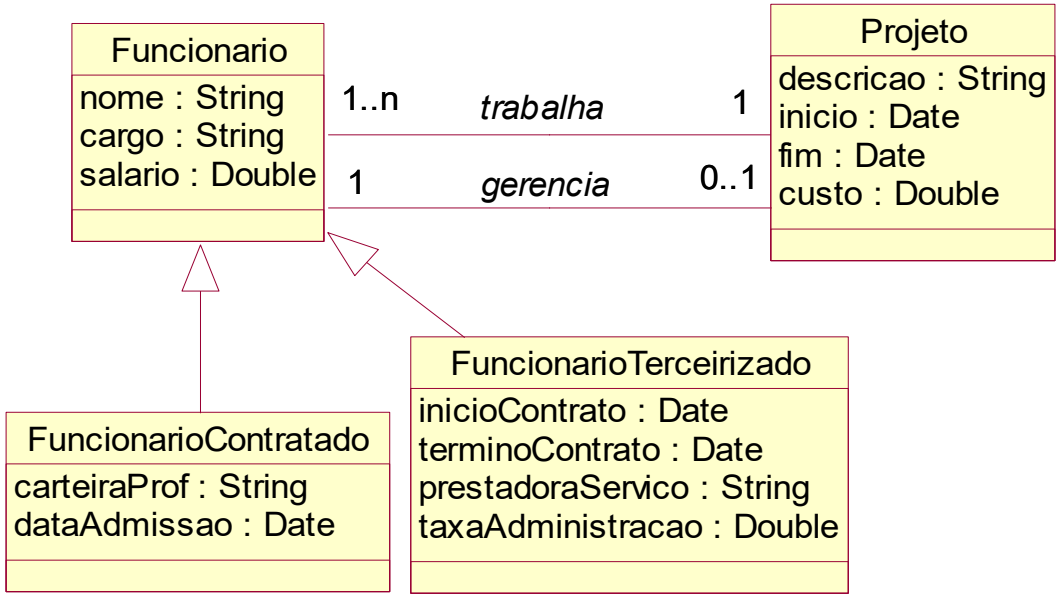
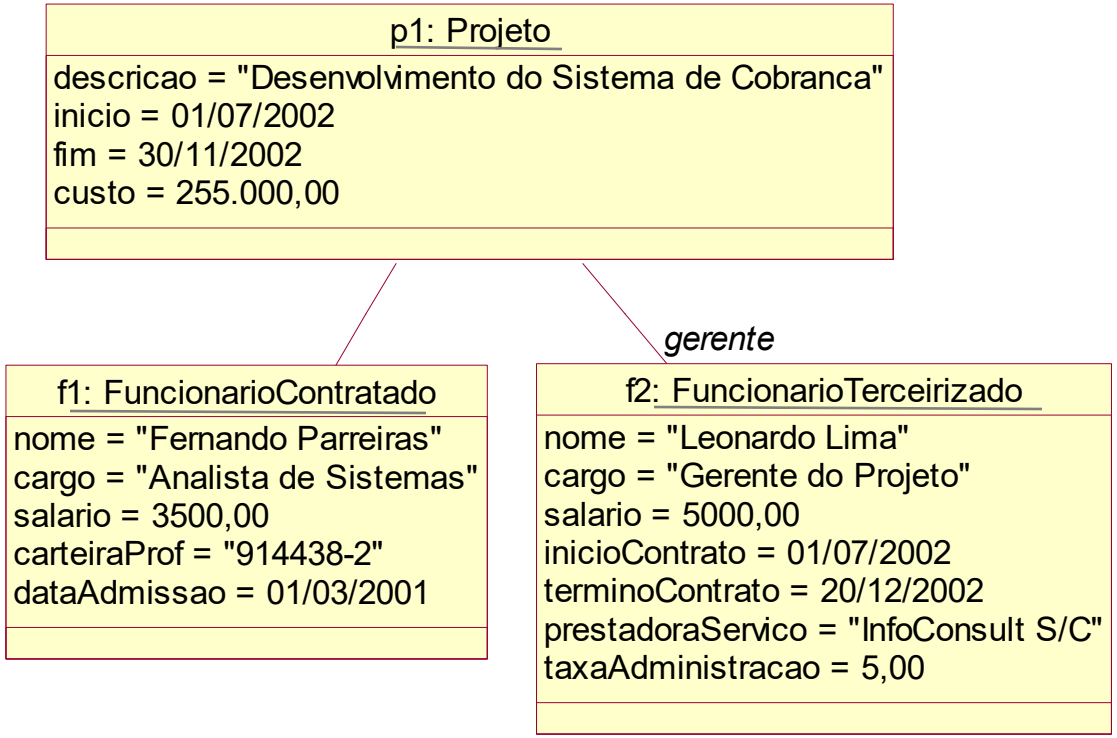


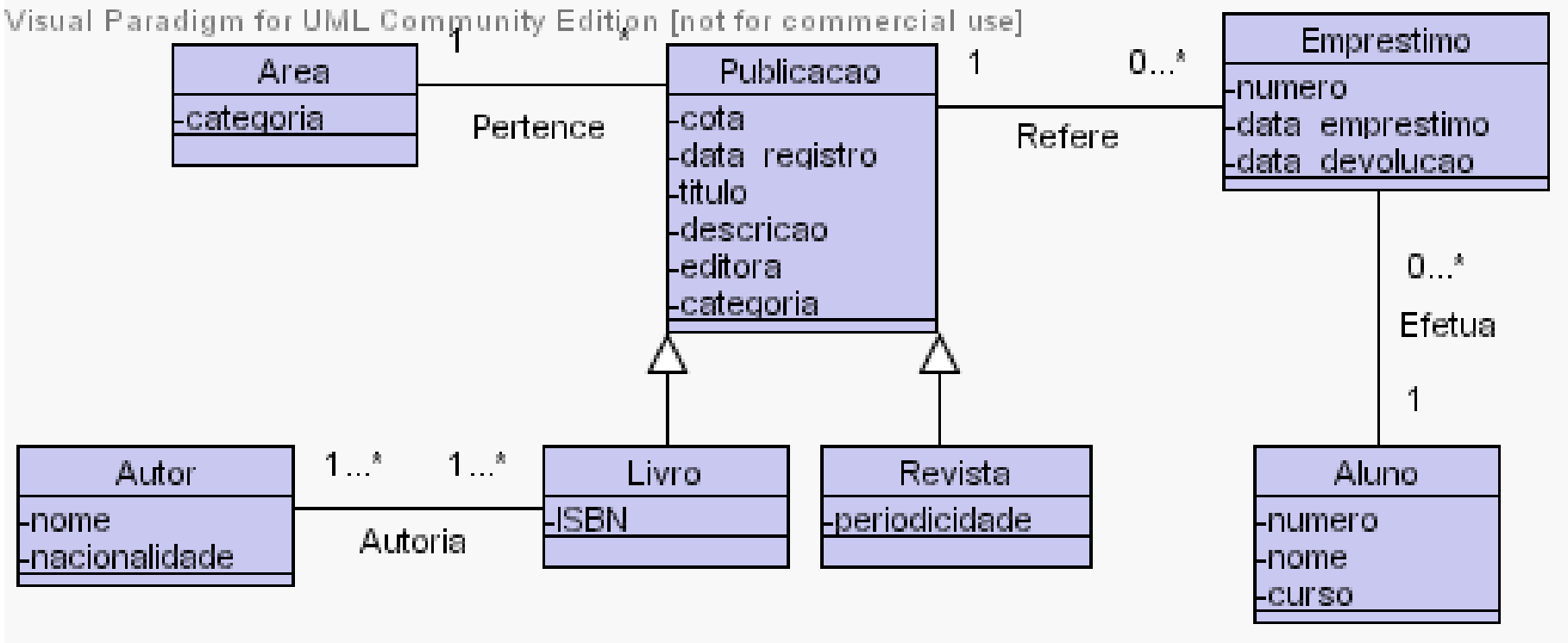
Diagrama de Classes e Diagrama de Objetos





Exercício: Biblioteca

- De acordo com o diagrama de Classes da Biblioteca, defina o diagrama de objetos para uma publicação do tipo Livro e outra do tipo Revista. É necessário que seja verificado o nome(s) do autor(es) da publicação.



eduardo.felipe@unifei.edu.br
fabiana.costaguedes@unifei.edu.br
rossana.junqueira@unifei.edu.br
wendelldiniz@unifei.edu.br